Roc-rk3399-pc

ARM Cortex-A72 + Cortex-A53

Embedded Development Platform

Development Manual

Rev. 1.0

2019/07/10

0755-86200561

support@100ask.net

Shenzhen, Guangdong, China

Shenzhen 100ask Technology Co.

注意事项与售后维修

1. 注意事项

- 使用产品之前，请仔细阅读本手册，并妥善保管，以备将来参考；
- 请注意和遵循标注在产品上的所有警示和指引信息；
- 请使用配套电源适配器，以保证电压、电流的稳定；
- 请在凉爽、干燥、清洁的地方使用本产品；
- 请勿在冷热交替环境中使用本产品，避免结露损坏元器件；
- 请勿在湿气过重、温度过高或过低环境中使用本产品，使用时注意产品的通风；
- 请勿将任何液体泼溅在本产品上，禁止使用有机溶剂或腐蚀性液体清洗本产品；
- 请勿在多尘、脏乱的环境中使用本产品，如果长期不使用，请包装好本产品；
- 请勿在振动过大的环境中使用，任何跌落、敲打或剧烈晃动都可能损坏线路及元器件；
- 请勿在通电情况下，插拔核心板及外围模块(特别是串口模块)；
- 请勿自行维修、拆解本产品，如产品出现故障应及时联系本公司进行维修；
- 请勿自行修改或使用未经授权的配件，由此造成的损坏将不予保修；

2. 售后维修

1) 保修期限

- 底板、核心板：三个月（非人为损坏）
- 显示屏：七天（非人为损坏）

2) 保修说明

- 7 天内：产品（底板、核心板、屏幕）有问题，本公司免费更换/维修，并承担来回运费；
- 7 天至 3 个月内：底板、核心板有问题，本公司免费维修，并承担来回运费（屏幕不提供维修）；
- 3 个月后：底板、核心板有问题，只收元器件价格维修，买家承担来回运费；

3) 联系方式

官方网站：www.100ask.net
淘宝网站：100ask.taobao.com

地　址：广东省深圳市龙岗区布吉南湾街道平吉大道建昇大厦 B1505
联系人：售后维修部
电　话：0755-86200561
邮　编：518114

邮寄须知：保修期限内，寄回本产品请预先垫付邮费，公司不接收任何到付快递。

技术支持与开发定制

1. 技术支持范围

1) 本公司提供的各类开发软件的安装，入门使用，环境搭建；
2) 本公司提供的所有裸机代码的烧写验证；
3) 本公司发布的操作系统的编译、烧写；
4) 本公司发布产品的工控板、模块的硬件原理；
5) 本公司发布的各种外设模块驱动及源码；
6) 本公司发布的配套手册在使用过程中遇到的问题；
7) 本公司产品的故障诊断及售后维修服务；

2. 技术讨论范围

由于嵌入式系统知识范围广泛，涉猎种类繁多，我们无法保证对各种问题都能一一解答，以下内容只提供技术讨论。

1) U-Boot 源码、Linux 内核源码的理解和修改；
2) 非本公司发布的 U-Boot、Linux 内核的编译和移植；
3) 非本公司发布的工控板的各类驱动支持；
4) 非本公司发布的外设模块的硬件原理和驱动设计；

3. 技术支持方式

1) 官方论坛发帖提问(推荐)：bbs.100ask.net
2) 官方淘宝通过阿里旺旺咨询：100ask.taobao.com
3) QQ 群咨询（QQ 群号咨询淘宝客服，需提供淘宝购买订单号验证加入）；
4) 技术支持邮箱：support@100ask.net
5) 联系电话：0755-86200561

4. 技术支持时间

星期一到星期五;上午 9:00—12:00;下午 14:00—17:30;

公司按照国家法定节假日安排休息，在此期间无法提供技术支持，请将问题发送至技术支持邮箱或在论坛对应板块发帖，我们将在工作日尽快给您回复。

5. 投诉和建议

如果您对我们有不满意或者建议，可发送邮件到 support@100ask.net 进行反馈，也可拨打 0755-86200561 取得联系，我们将不断改进。

6. 定制开发服务

本公司提供嵌入式操作系统底层驱动、硬件板卡的有偿定制开发服务，以缩短您的产品开发周期。请将需求发送邮件到 support@100ask.net。

资料获取与后续更新

1. 资料的获取

1) 百度网盘下载

百度网盘里面有本产品的所有配套资料，包括原理图、发布的 U-Boot、内核镜像和源码、所需的开发软件、工具等等。

进入 www.100ask.net，在导航栏选择“资料下载”，点击“到百度网盘下载”，跳转到百度网盘后，找到对应的文件夹即可。

2) 视频配套教程

后续会为该工控板录制一套裸机、Linux 驱动、应用的配套付费教学视频，有需要的客户可以进入官方淘宝 100ask.taobao.com 选购。

3) 维基百科教程

维基百科里面会有视频配套的笔记，进入 wiki.100ask.net，选择对应的板块查看。

2. 后续更新

后续文档、视频等资料的更新，为了确保您的资料是最新状态，请密切关注我们的动态，我们将会通过微信公众号和 QQ 群公告推送，购买了本产品的客户请添加 QQ 群（QQ 群号咨询淘宝客服，需提供淘宝购买订单号验证加入）或关注微信公众号。

版权声明

百问科技©2019

深圳百问网科技有限公司版权所有，并保留对本手册及声明的一切权力。

未得到本公司的书面许可，任何单位和个人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言、将其全部或部分用于商业用途

更新记录

类别	100ASK_ROC-RK3399-PC 文档
文档名	Roc-rk3399-pc Development Manual
当前版本	1.0
适用型号	ROC-RK3399-PC
编辑	百问科技文档编辑团队
审核	韦东山

修改日志		
版本	修改时间	更改说明
1.0	2019.03.18	初始版本

目录

注意事项与售后维修 I

技术支持与开发定制 II

资料获取与后续更新 III

版权声明 IV

更新记录 V

目录 1

第 1 章 软件资源介绍 3

1.1 bootloader 介绍 3

1.2 linux 内核介绍 3

1.3 根文件系统介绍 4

1.4 开发工具介绍 4

第 2 章 搭建开发环境 6

2.1 使用 vmware 运行虚拟机 6

2.2 安装、使用 MobaXterm 8

2.3 使用 FileZilla 新建 SFTP 连接 13

2.4 安装使用 SourceInsight 14

第 3 章 构建系统 21

3.1 获取源码 21

3.2 配置交叉编译工具链 21

3.3 解压编译 bootloader 22

3.4 编译 linux kernel 24

3.5 构建根文件系统 25

第 4 章 烧写系统 27

4.1 SD 卡启动 27

4.2 NFSROOT 启动 29

第 5 章 平台功能测试 30

5.1 网卡接口测试 31

5.2 USB Host 接口测试 33

5.3 OTG 接口测试.....34
5.4 RTC 测试.....36
5.5 按键测试.....37
5.6 LED 灯测试.....38
5.7 串口测试.....38

第1章软件资源介绍

简介:以下资源仅限于在百问网官方购买及百问网授权店铺购买的开发板上使用，我们在官方的基础上做了大量的定制化修改，精简了系统，简化下载以及编译流程，更大的提升开发效率，系统不同于 firefly 官方。如需为 roc-rk3399-pc 编译 ubuntu 以及 Android 系统请参考 http://wiki.100ask.org/100ask_roc-rk3399-pc 编译 firefly linux-sdk 系统，编译 android 固件，章节内容。

1.1 bootloader 介绍

Bootloader 是在操作系统运行之前运行的一段代码，用于引导操作系统。通常每个操作系统都有一组专属的引导加载程序。引导加载程序通常可以通过多种方式引导操作系统内核，还有各种命令用于调试或修改内核运行环境。

bootloader有许多版本，这里我们选择使用Das U-Boot（“Universal BootLoader”，通常缩写为U-Boot）是一个开源的主引导加载程序，用于引导设备的操作系统内核，并含有多种命令以便调试系统。它适用于多种计算机体系结构，包括 68k，ARM，Blackfin，MicroBlaze，MIPS，Nios，SuperH，PPC，RISC-V 和 x86。

- U-boot 官网 https://www.denx.de/wiki/U-Boot
- 源码下载页面 http://ftp.denx.de/pub/u-boot/
- Git 仓库地址 https://git.dev.tencent.com/weidongshan/100ask_roc-rk3399-pc_uboot.git

百问网提供的 U-Boot 有如下特性：

特性	是否支持	作用	是否开源
sd 卡启动	支持	通过 sd 卡启动系统	Yes
网卡启动	支持	通过网络挂载方式启动系统	Yes

注意: u-boot 编译方法请参 configbootloader

1.2 linux 内核介绍

Linux 内核（英语：Linux kernel）是一种开源的类 Unix 操作系统宏内核。整个 Linux 操作系统家族基于该内核部署在传统计算机平台（如个人计算机和服务器，以 Linux 发行版的形式）和各种嵌入式平台，如路由器、无线接入点、专用小交换机、机顶盒、FTA 接收器、智能电视、数字视频录像机、网络附加存储（NAS）等。工作于平板电脑、智能手机及智能手表的 Android 操作系统同样通过 Linux 内核提供的服务完成自身功能。尽管于桌面计算机的占用率较低，但基于 Linux 的操作系统统治了几乎从移动设备到主机的其他全部领域。

- Linux kernel 官网 https://www.kernel.org/
- linuxKernel 维基百科 https://www.wiki.kernel.org/
- 在线阅读 linuxkernel 源码 https://elixir.bootlin.com/
- TI 官方 linuxkernel 参考文档 http://processors.wiki.ti.com/index.php/Linux_Kernel_Users_Guide
- Git 仓库地址 https://git.dev.tencent.com/weidongshan/100ask_roc-rk3399-pc_kernel.git

上述 Git 仓库是专为 100ask_roc-rk3399-pc 开发板制定的 Linux 内核，它有如下特性：

名称	是否支持	作用	是否开源
USB HOST	支持	USB Host 驱动，支持 OHCI 和 EHCI 两种传输模式 YES	Yes

USB OTG	支持	Usb otg 驱动，支持 device 模式和 host 模式	Yes
NetworkPort	支持	RJ-45 以太网驱动	Yes
MMC/SD	支持	MMC/SD 卡驱动	Yes
EMMC	支持	EMMC 驱动	Yes
I2C	支持	I2C 驱动	Yes
SPI	支持	SPI 驱动	Yes
RTC	支持	内置 RTC 时钟驱动	Yes
ADC	支持	ADC 驱动	Yes
UART	支持	串口驱动	Yes
PMU	支持	电源管理驱动	Yes
LED	支持	LED 驱动，包括 GPIO LED 和 PWM	Yes
Button	支持	GPIO Button 驱动	Yes
Camera	支持	摄像头驱动	Yes

注意：内核编译方法请参编译 linuxkernel

1.3 根文件系统介绍

Buildroot 是一组 Makefile 和补丁，可简化和自动化为嵌入式系统构建完整且可启动的 Linux 环境的过程，同时使用交叉编译允许在单个基于 Linux 的开发系统上构建多个目标平台。Buildroot 可以自动构建所需的交叉编译工具链，创建根文件系统，编译 Linux 内核映像，并生成引导加载程序对于目标嵌入式系统，或者它可以执行这些步骤的任何独立组合。例如，可以单独使用已安装的交叉编译工具链，而 Buildroot 仅创建根文件系统。

参考网址

- Buildroot 用户手册 https://buildroot.org/downloads/manual/manual.html
- BuildRoot 源码下载位置 https://buildroot.org/downloads/
- 目录结构的位置以及作用请参考网址 http://wiki.100ask.net/Buildroot
- Git 仓库地址 https://git.dev.tencent.com/weidongshan/100ask_roc-rk3399-pc_buildroot.git

文件系统支持	作用
web 环境	html+php 网络开发环境
Vim 开发环境	vim8.0 环境
串口传输文件	通过 z-mode 协议使用串口传输文件到开发板
ssh 工具	通过 ssh 工具登录开发板系统
nfs 工具	通过 nfs 命令挂载主机文件
电子相册	在 LCD 上浏览图片，支持放大缩小等
Arduino 编程	在 windows 下对开发板进行模块编程

注意：根文件系统源码下载及编译方法请参构建根文件系统

1.4 开发工具介绍

开发所涉及的所有工具，名称以及文件存放位置，还有软件作用简单介绍，如下表格所示

名称	位置	软件作用
arduino-1.8.9-windows.exe	01_tools\	Windows 下编译开发环境，主要适用模块应用开发
FileZilla_3.22.1_win32-setup.exe	01_tools\	Windows 下 ftp 工具，传输文件到 Ubuntu 系统中
HPUSBDiskStorage_Format_Tool 2.0.6.EXE	01_tools\	Windows 下格式化 sd 卡工具，主要适用于在烧写 nandflash 启动进行格式化
MobaXterm_Portable_v11.0.zip	01_tools\	Windows 下 ssh 工具;ftp 工具;串口工具。
npp.7.6.1.Installer.exe	01_tools\	Window 下常用的文本阅读工具
qt-creator-opensource-linux-x86_64-4.8.0.run	01_tools\	Linux 下 qt 应用程序开发客户端
SD_CardFormatter5.0.1Setup.exe	01_tools\	Windows 下 sd 卡格式化工具，用于 sd 卡启动
sourceinsight4098-setup.exe	01_tools\	Windows 下源码阅读工具
100ask-vmware_ubuntu18.04.7z	01_tools\	ubuntu18.04 系统，用于开发使用
win32diskimager-1.0.0-install.exe	01_tools\	Windows 下开发板系统镜像烧写工具，用于 sd 卡启动
VMware-workstation-full-15.1.0-13591040.exe	01_tools\	Windows 下系统虚拟化工具，用于虚拟化 ubuntu 18.04 系统

第2章搭建开发环境

本章主要讲解如何使用 vmware Workstation Player 加载百问网制作的 ubuntu18.04 虚拟机镜像，并使用 mobaXterm 工具建立 ssh 连接，FTP 连接以及串口连接；介绍代码阅读工具 sourceinsight 如何新建一个内核项目，最后是关于 ubuntu18.04 的一些配置说明

本章所需工具

- VMware-workstation-full-15.1.0-13591040.exe
- sourceinsight4098-setup.exe
- MobaXterm_Portable_v11.0.zip

2.1 使用 vmware 运行虚拟机

2.1.1 安装wmwareWorkstationPlayer

Windows 下有很多虚拟机软件，目前市面上流行的有 VMware 和 VirtualBox。VMware 分为收费专业版 Workstation Pro 和非商用免费版 Workstation Player，推荐使用 Workstation Player。

首先从 VMware 官网（www.vmware.com）下载 Workstation Player 安装包或者使用我们提供的安装包(目录在资料光盘\01_tools 目录下)，这里使用资料光盘安装包：VMware-workstation-full-15.1.0-13591040.exe。

VMWare 安装完成后，有两个软件，它们都可以使用，建议使用第 2 个：

1. Vmware Workstation Pro：这是收费的，可以试用 30 天。
2. Vmware Workstation 15 Player：这是免费的。

百问网团队特意根据本节内容录制了 1 节视频。

腾讯视频在线观看：https://v.qq.com/x/page/w05186fncbn.html

注明：如需自行配置 ubuntu 等开发环境请参考 http://wiki.100ask.net/Configuring_ubuntu

2.1.2 使用虚拟机打开Ubuntu

解压资料光盘提供的虚拟机压缩包(资料光盘\ 01_tools \ 100ask-vmware_ubuntu18.04.7z) 如图 2.1.2.1 所示。

注意:

- 100ask-vmware_ubuntu18.04 不支持 **win7 32** 位和 **Windows xp** 电脑系统，有疑问请联系技术支持。
- 100ask-vmware_ubuntu18.04 用到了 **Virtualization Technology(VT)**技术,请提前把电脑的 VT 打开，由于不同的电脑打开方法不一样，所以具体的打开方法请百度。

1) 运行 Vmware Workstation 15 Player，在欢迎界面选中”**打开虚拟机**”，如图 2.1.2.2 所示。

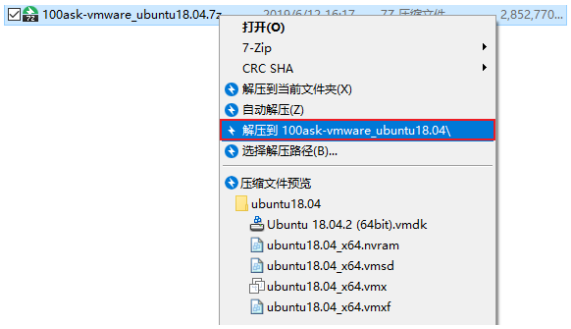


图 2.1.2.1 解压虚拟机

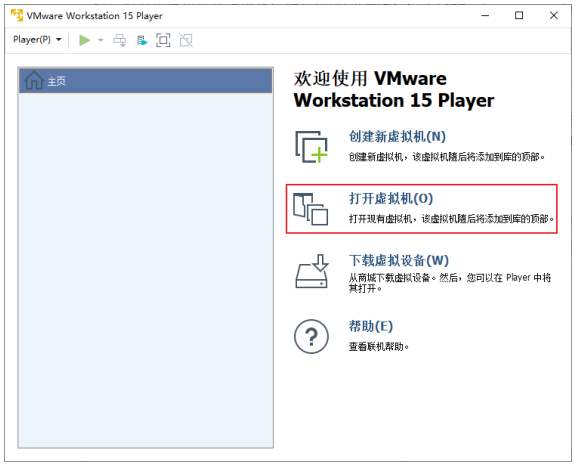


图 2.1.2.2 打开虚拟机

2) 在弹出的窗口中，找到前面解压的 Ubuntu，选中“**ubuntu-18.04_x64.vmx**”,再单击“**打开**”如图 2.1.2.3 所示。再单击“**开启此虚拟机**”，如图 2.1.2.4 所示。

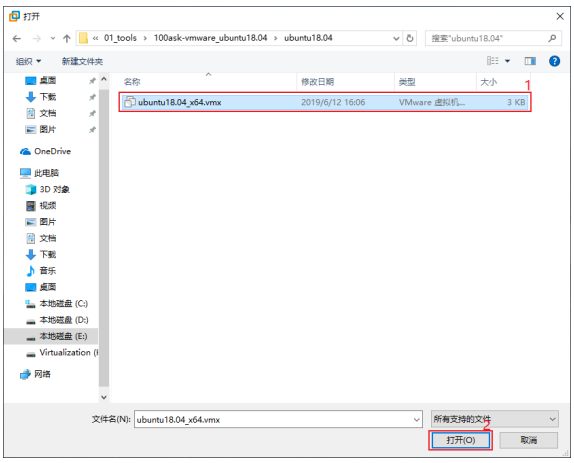


图 2.1.2.3 选中 Ubuntu

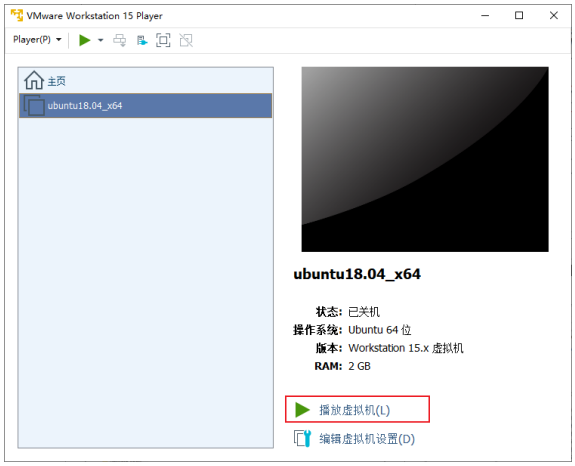


图 2.1.2.4 播放虚拟机

3) 第一次打开会有如图 2.1.2.5 所示的提示，选择默认的“**我已复制该虚拟机**”。此时 VMware Workstation 开始启动虚拟机，等待一会后出现如图 2.1.2.6 所示的登陆界面，在“**Password**”中输入密码“**123456**”回车（注意：虚拟机默认没有开启小键盘，如果使用小键盘输入，请先开启小键盘）。



图 2.1.2.5 我已复制该虚拟机

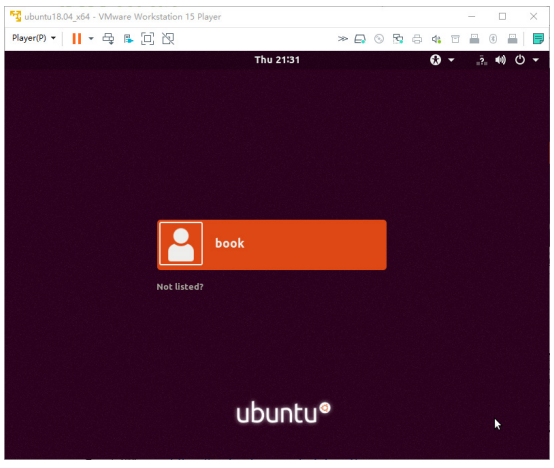


图 2.1.2.6 登陆界面

2.2 安装、使用 MobaXterm

MobaXterm 是您远程计算的终极工具箱。在单个 Windows 应用程序中，它提供了大量功能，这些功能是为程序员、网站管理员、IT 管理员以及需要以更简单的方式处理远程作业的所有用户量身定制。MobaXterm 向 Windows 桌面提供所有重要的远程网络工具（SSH，X11，RDP，VNC，FTP，MOSH .....）和 Unix 命令（bash，ls，cat，sed，grep，awk，rsync）等，仅仅需要一个 exe 文件即可。

官网　https://mobaxterm.mobatek.net/
使用文档 https://mobaxterm.mobatek.net/documentation.html

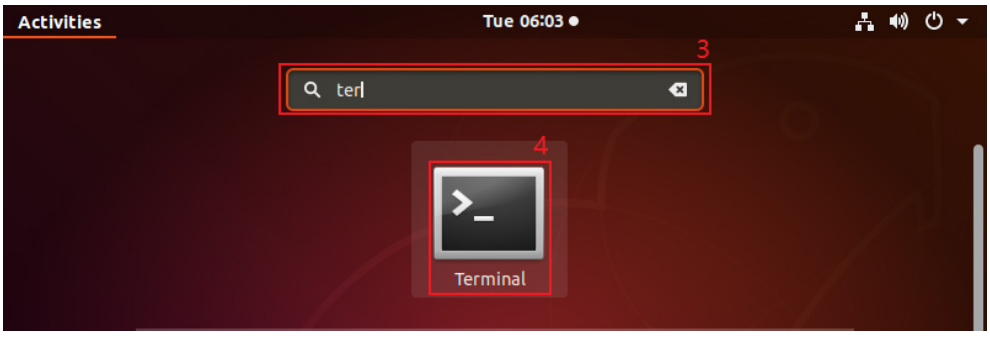
2.2.1 安装及获取ubuntuIP地址

解压文件“MobaXterm_Portable_v10.4.zip”即可。第一次打开会自解压比较慢，后续就正常了。

使用 MobaXterm 是为了远程登录 Vmware 里的 Ubuntu，所以需要先知道 Ubuntu 的 IP。方法如下图 2.2.1.1 所示。点击 Ubuntu 桌面左上角图标，输入“**term**”可以得到图中蓝框中的“**Terminal**”程序，运行它；然后执行“**ifconfig**”命令即可查看 Ubuntu 的 IP，如图 2.2.1.2 所示。



图 2.2.1.1 打开 Terminal



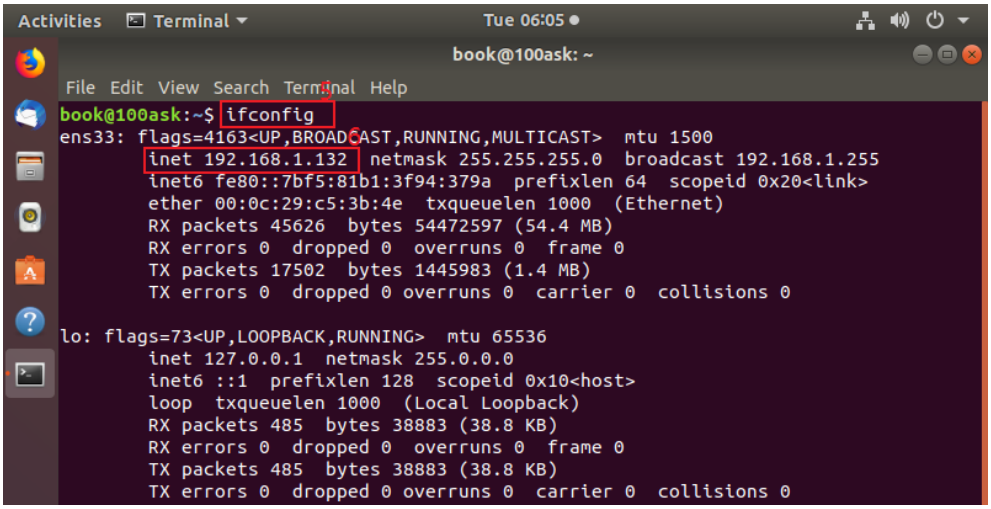


图 2.2.1.2 查看 Ubuntu 的 IP

注意：本文档假设 Ubuntu 的 IP 为 192.168.1.132，在上图中网卡为 ens33(**你的电脑上网卡名也许不一样**)。

2.2.2 新建SSH连接

1) 打开 MobaXterm，点击左上角的 **Session**（会话控制），在弹出的窗口中选择 SSH，如图 2.2.2.1 和 2.2.2.2 所示。

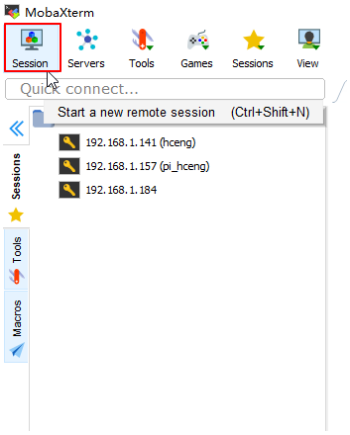


图 2.2.2.1 选择 Session

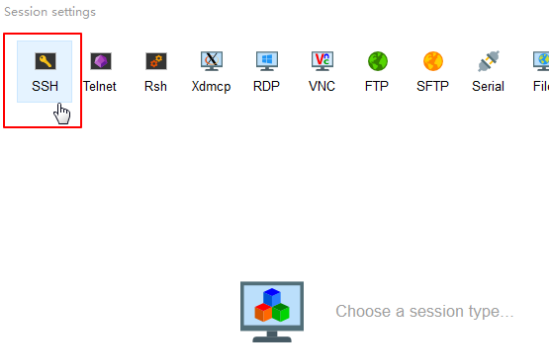


图 2.2.2.2 选择 SSH

2) 在弹出操作框里输入 Ubuntu 的 IP 和端口号（默认是 22），然后点击“**OK**”，如图 2.2.2.3 所示。

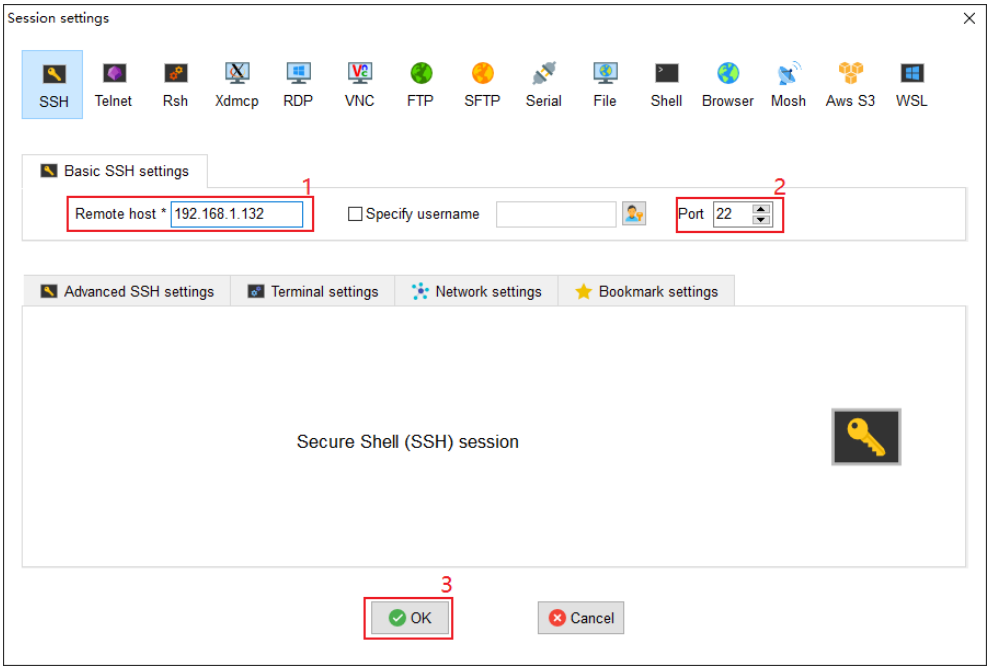


图 2.2.2.3 SSH 选项

3) 在新窗口中输入账号“**book**”和密码“**123456**”，再按下键盘“回车键”登陆 Ubuntu。此时界面分为两块，左边的是主机的文件，右边是终端。勾选左下角的“**Follow terminal folder**”可以让它们的工作路径保持一致，如图 2.2.2.4 所示。

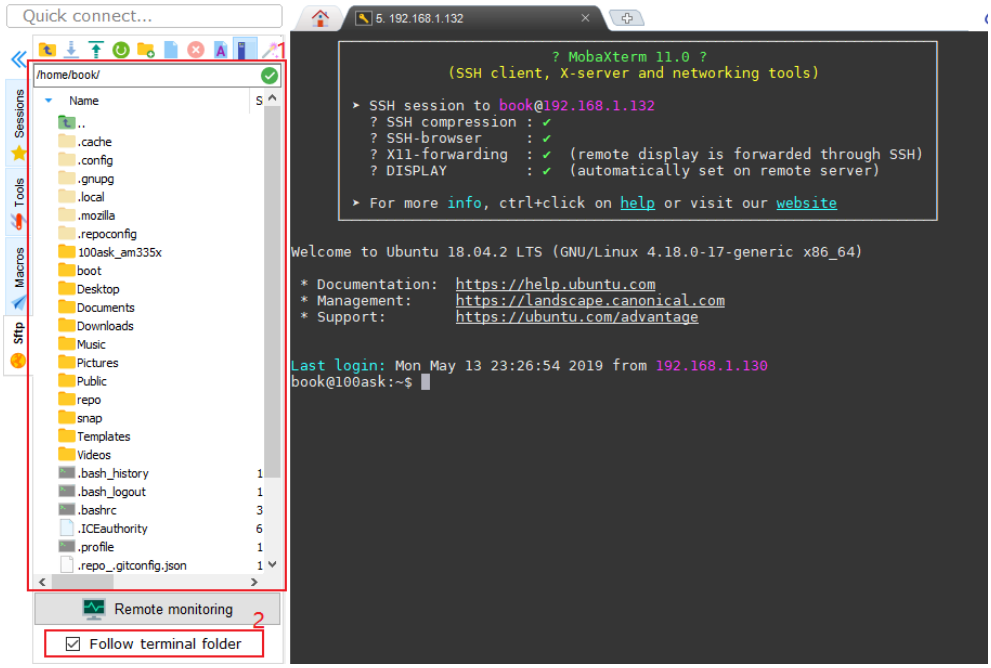


图 2.2.2.4 SSH 工作窗口

2.2.3 新建串口连接

在后面的操作里，都是通过串口与板子进行“交流”。串口是串行接口的简称，是指数据一位一位地顺序传送，其特点是通信线路简单。

在电脑上安装好 MobaXterm 后，接上 USB 串口模块，并跟开发板连好线。

在 MobaXterm 里敲打键盘，就会通过 USB 串口模块，将数据经过 TTL 延长线传给板子，板子就能接收到我们在电脑上发送的数据。

反过来，板子发送的数据首先经过 TTL 延长线到 USB 串口模块，MobaXterm 读取数据后显示出来。

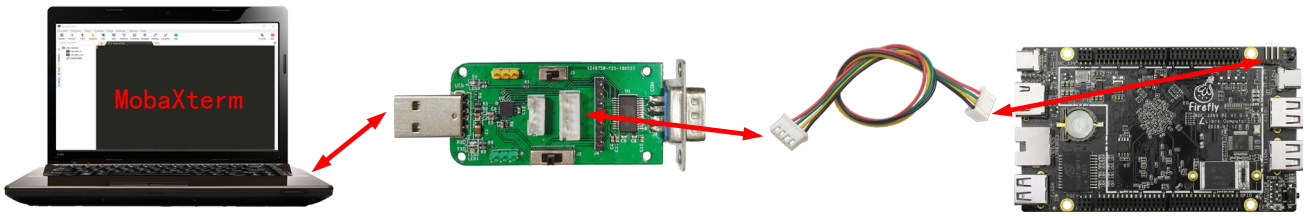


图 2.2.4.1 串口连接示意图

1) 安装 USB 串口模块驱动

将 USB 转 RS232/TTL 串口模块插在电脑 USB 上，此时 Windows 会自动安装驱动(安装可能比较慢，等一分钟左右)。打开电脑的“设备管理器”，在“端口 (COM 和 LPT)”项下，可以看到如图 2.2.4.2 中的“USB Serial Port(COM3)”。这里的“**COM3**”可能与你电脑上的不一样，记住你电脑显示的数字。

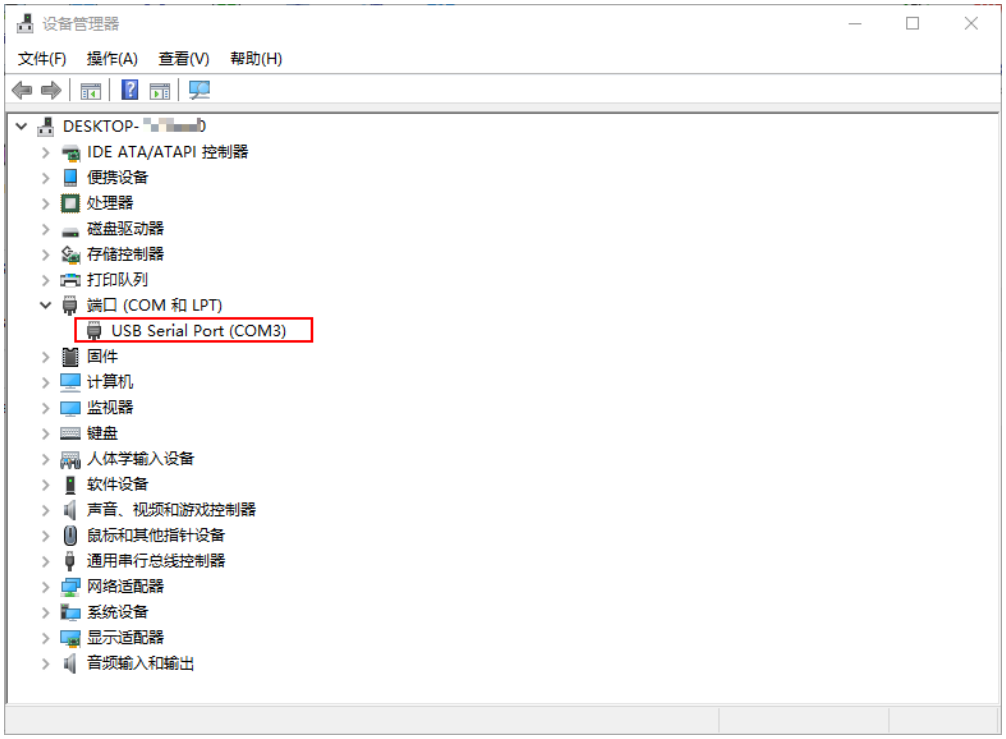


图 2.2.4.2 设备管理器查看串口端口号

如果电脑没有显示出端口号，就需要手动安装驱动，从驱动精灵官网（www.drivergenius.com）下载一个驱动精灵，安装、运行、检测，会自动安装上串口驱动。

2) 连线、配置

首先如图 2.2.4.3 所示将串口模块与电脑、板子连接。

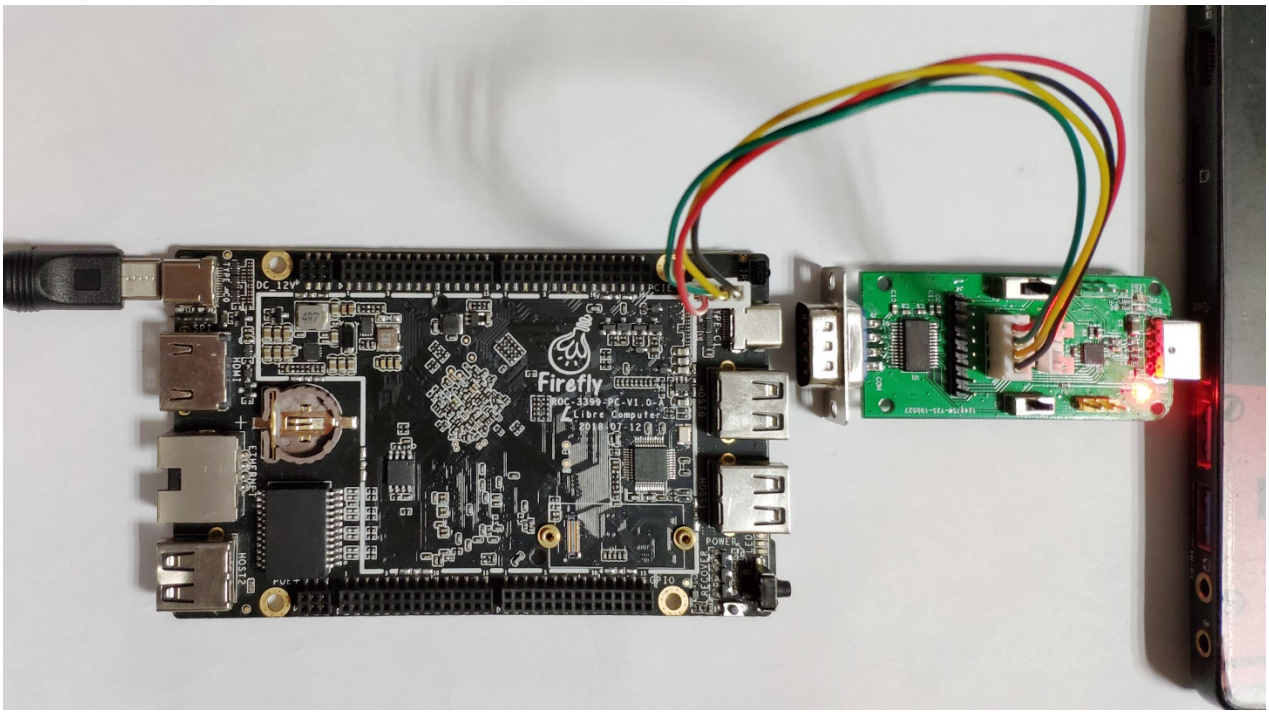


图 2.2.4.3 串口模块连接实物图

其中特别需要注意的几点：

a) 串口模块的电平选择开关拨到如图所示的右边，表示切换到 TTL 电平；
b) 串口模块附赠两条排线，使用较宽的那根(2.54mm 间距)连接串口模块最中间接口；
c) 排线的另一端插在 ROC-RK3399-PC 如图所示位置，注意黑线(GND)在图中最右边；
d) 该处没有 eMMC，在该处背面的 TF 卡槽里，需要插上烧写好系统的 TF 卡；
e) 板子如图所示准备好配套的电源，注意是在图中的左边 Type C 接口，因为没有电源开关，插上就自动启动了，所以暂时先不插电源；

打开 MobaXterm，点击左上角的“Session”，在弹出的界面选中“Serial”，如图 2.2.4.4 所示选择端口号（前面设备管理器显示的端口号 COM3）、波特率（Speed 115200）、流控（Flow Control: none）,最后点击“OK”即可。

注意：流控（Flow Control）一定要选择 none，否则你将无法在 MobaXterm 中向串口输入数据。

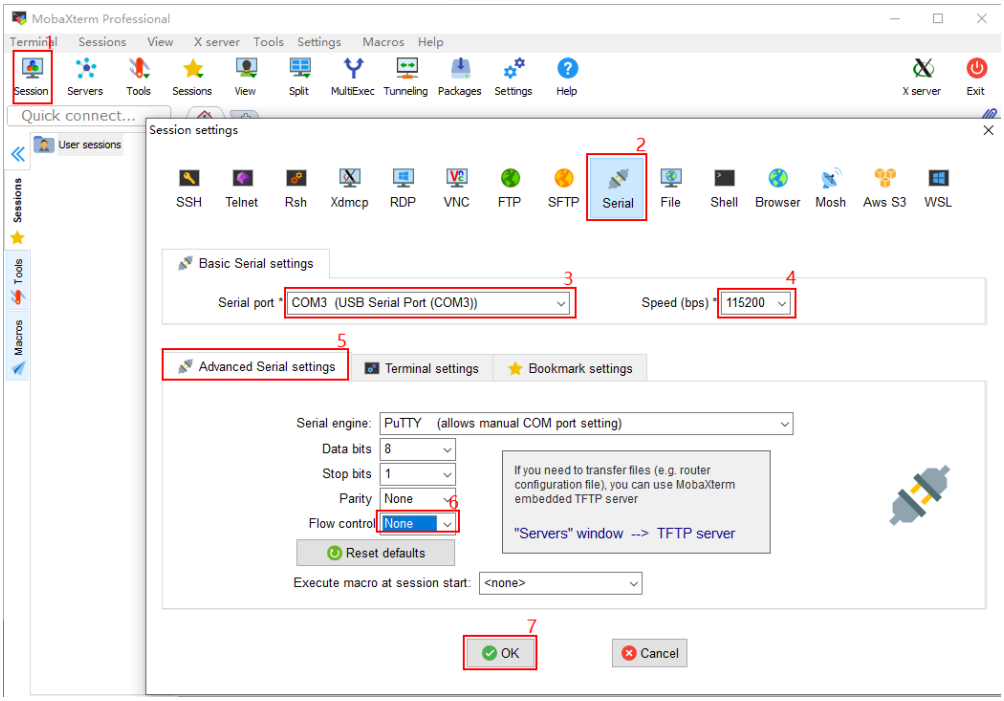


图 2.2.4.4 MobaXterm 串口对话设置

随后显示一个黑色的窗口， 此时打开板子的电源开关，将收到板子串口发过来的数据，如图 2.2.4.5 所示。

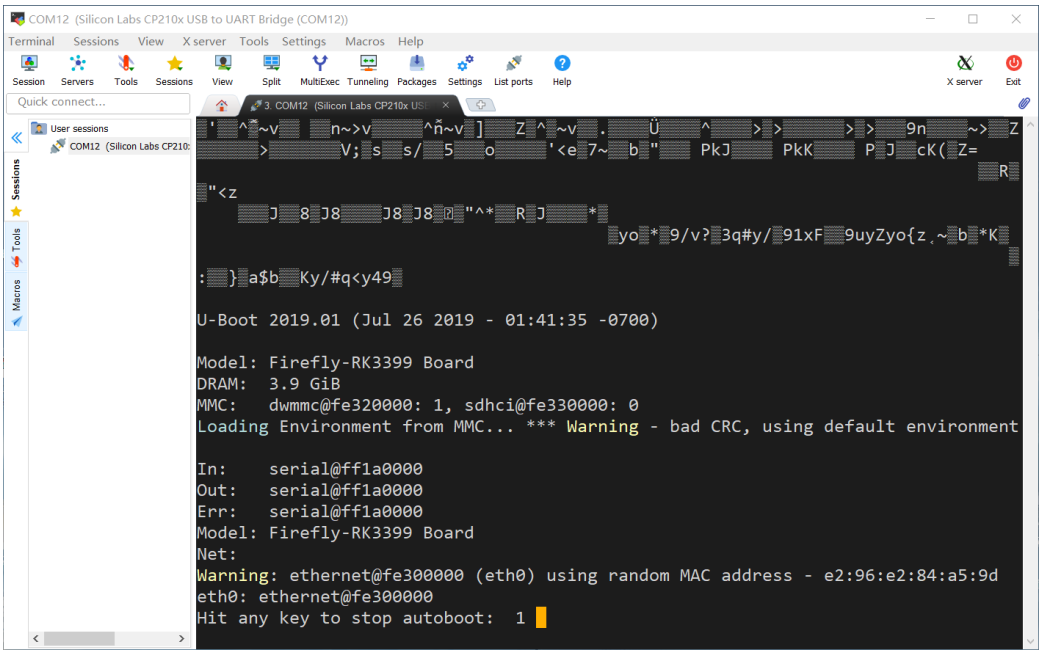


图 2.2.4.5 接收板子串口发送的数据

2.3 使用 FileZilla 新建 SFTP 连接

MobaXterm 支持 FTP 和 SFTP 连接。FTP 安全性没有 SFTP 好，但速度比 SFTP 快，可根据自己需求选择适当的协议。MobaXterm 的 FTP、SFTP 有时不稳定，我们推荐使用另一个软件 FileZilla 来传输文件。

注意：我们提供的 ubuntu 中没有安装 FTP 服务，你只能使用 SFTP 服务；后文提到的 TFTP 服务，跟 FTP 服务是不一样的，U-Boot 可以使用 TFTP 从 Ubuntu 下载文件。

1) 使用 FileZilla 软件来实现 SFTP 文件传输：

安装、运行 FileZilla 后，如图 2.2.3.1tu 的 IP（192.168.1.132、用户名（book）、端口号（22），单击“快速链接”即可。

注意：端口号必须填入 22，它对应 SFTP 服务。

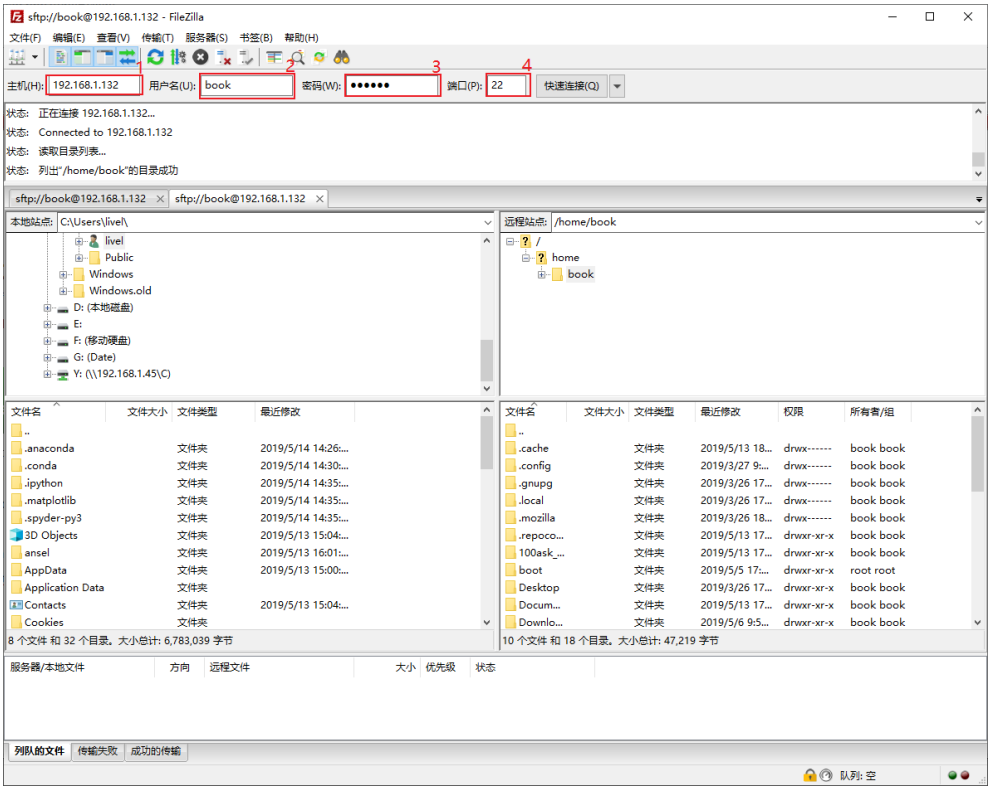


图 2.3.1 FileZilla 连接成功

2.4 安装使用 SourceInsight

2.4.1 SourceInsight简介

Source Insight 是 Source Dynamics 的源代码编辑器。Source Insight 提供语法突出显示，代码导航和可自定义的键盘快捷键。它不仅仅是一个编辑器，而是一个理解大型源代码库的工具，因此被称为“程序编辑器和分析器”。它灵活轻便，提供有用的功能，如关系，上下文和符号窗口。它还可以显示引用树，类继承图和调用树，因为它在自解析源时构建了符号信息的内部数据库。它的最大好处是加快了对不熟悉项目的代码理解。

参考网址

- 官网主页 https://www.sourceinsight.com/
- 软件下载页面 https://www.sourceinsight.com/trial/
- 用户使用教程 https://www.sourceinsight.com/doc/v4/userguide/index.html

2.4.2 安装并新建项目示例

安装并运行(资料光盘\ 01_tools)中的 sourceinsight4098-setup.exe 文件。

这里我们新建一个 linux kernel 的 sourceInsight 为例对 sourceInsight 使用进行说明

1) 在桌面上找到 sourceInsight 双击运行，点击 **Project->New Project**,如图 2.3.2.1 所示

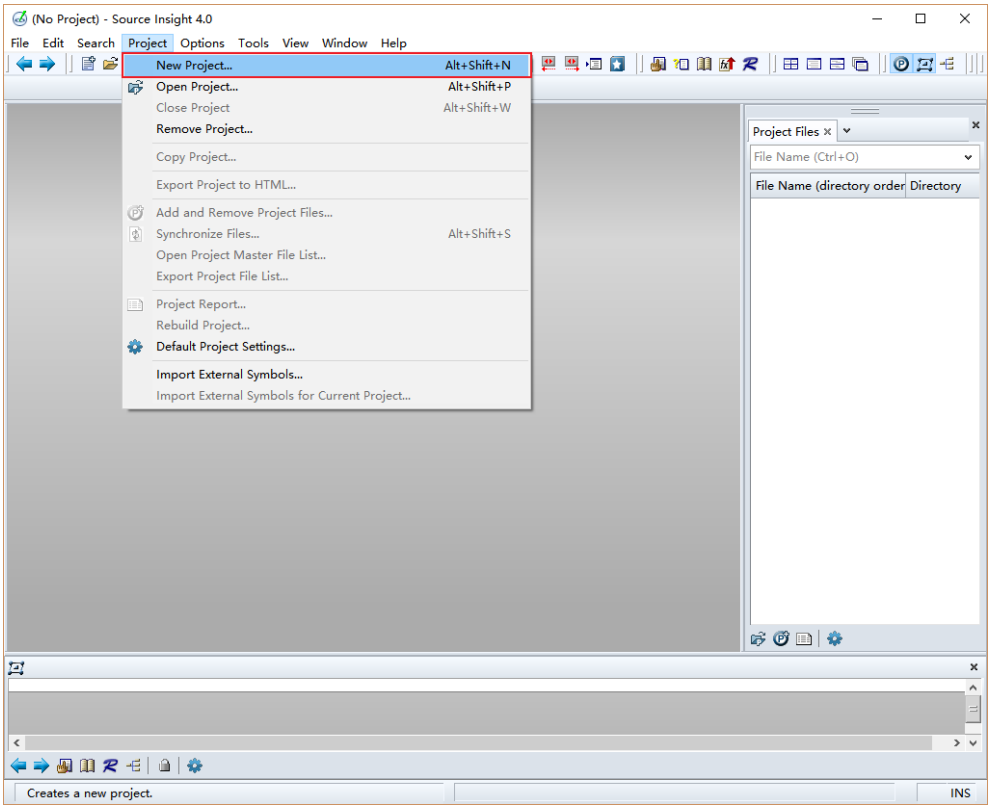


图 2.3.2.1 选择 New Project

2) 在弹出的 New Project 对话框中设置 **New project name**(项目的名称)让后设置 **Where do you want to store the project data file?** (项目文件保存位置)，点击 **Browse...**选择所在保存位置，设置好以后点击 OK 如图 2.3.2.2 所示

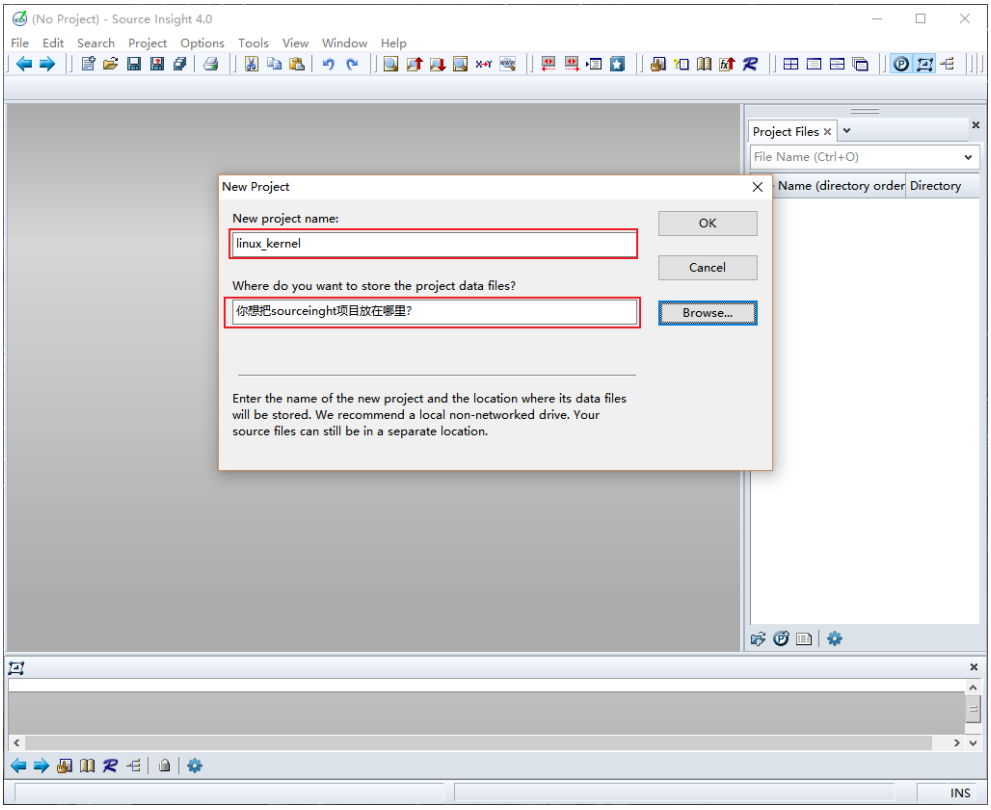


图 2.3.2.2 选择项目文件保存位置

3) 设置 **Project Source Directory – the main location of your source files**: (项目源目录 - 源文件的主要位置:)也就是内核源码所在目录，点击红框左边 … 选择源码目录，点击 OK

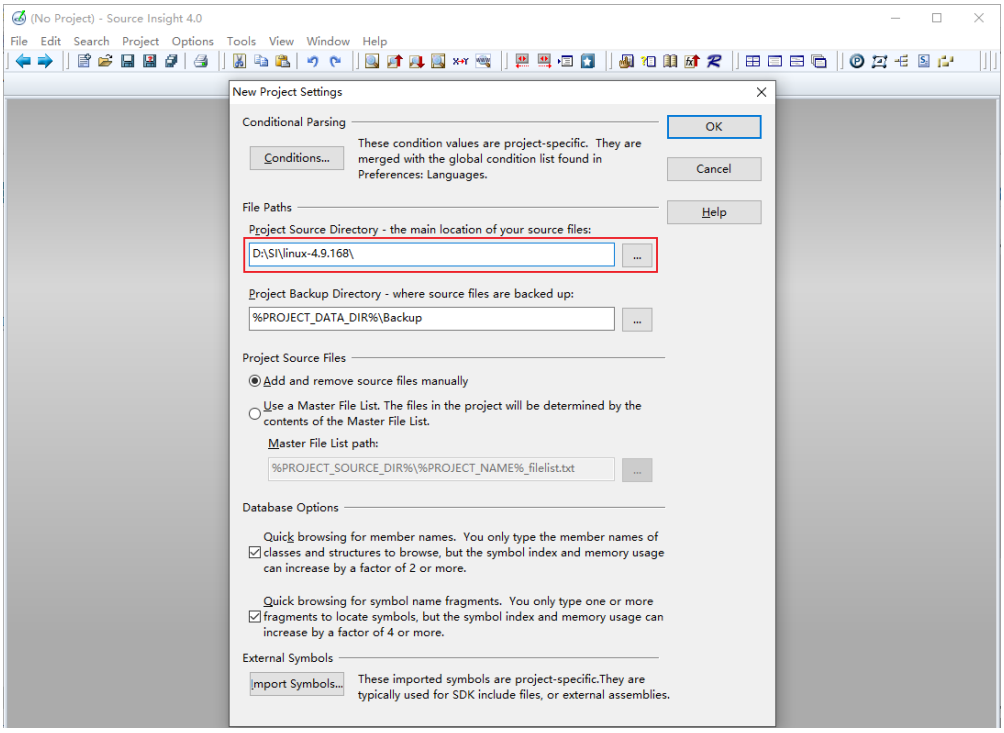
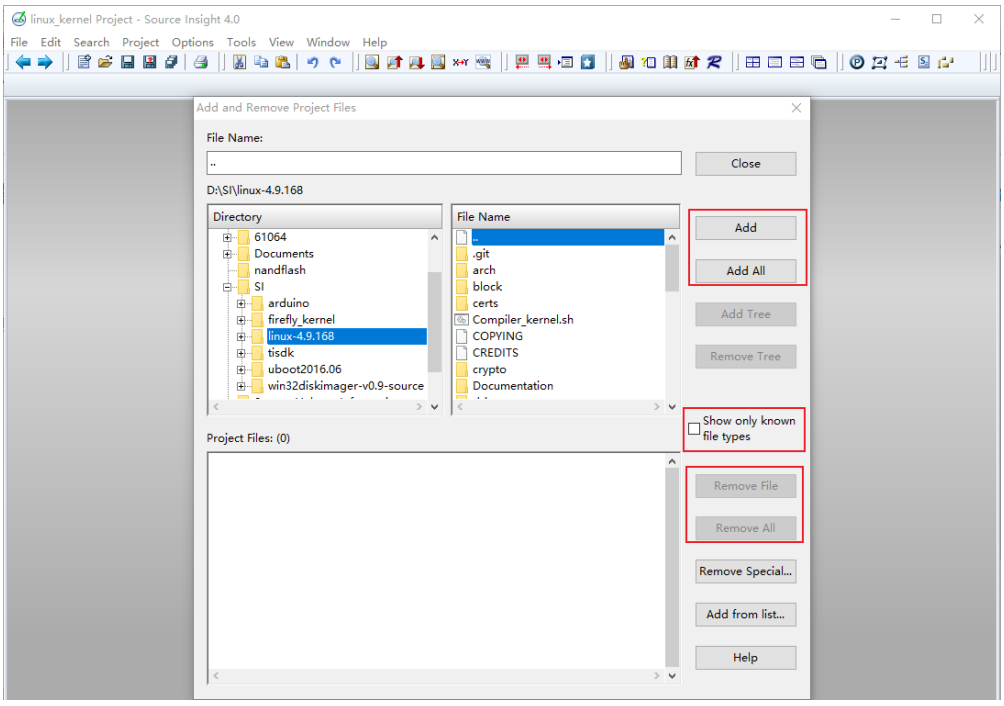


图 2.3.2.3 选择源码所在位置

4) 添加源代码，点击 Add 或 Add All; 其中 Add 是手动选择需要添加的文件，而 Add All 是添加所有文件，一般我们选择 **Add All**,在弹出的提示框中选中 **Recursively add lower sub-directories**(递归添加下级的子目录)点击 **OK**。同样的 Remove File,Remove All 是移除单个文件或者移除所有文件



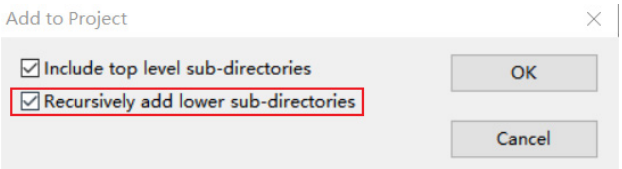


图 2.3.2.4 添加源码文件

5) 添加文件完成后点击 Close，此时界面会返回到主界面

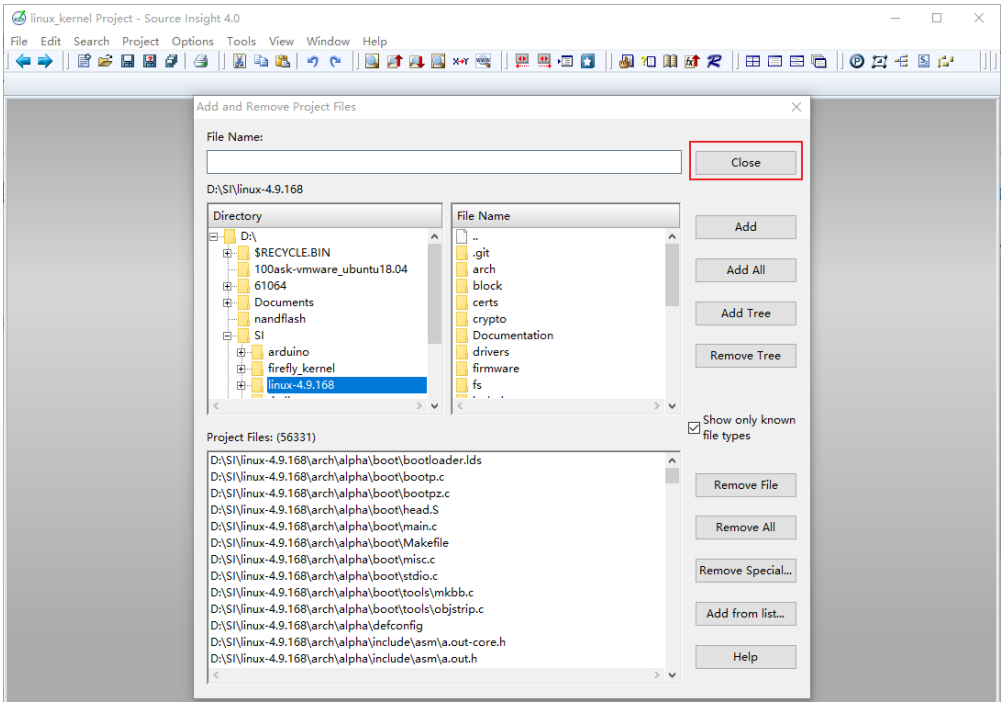


图 2.3.2.5 关闭添加源码窗口

6) 接下来我们需要点击 Synchronize File(同步文件)

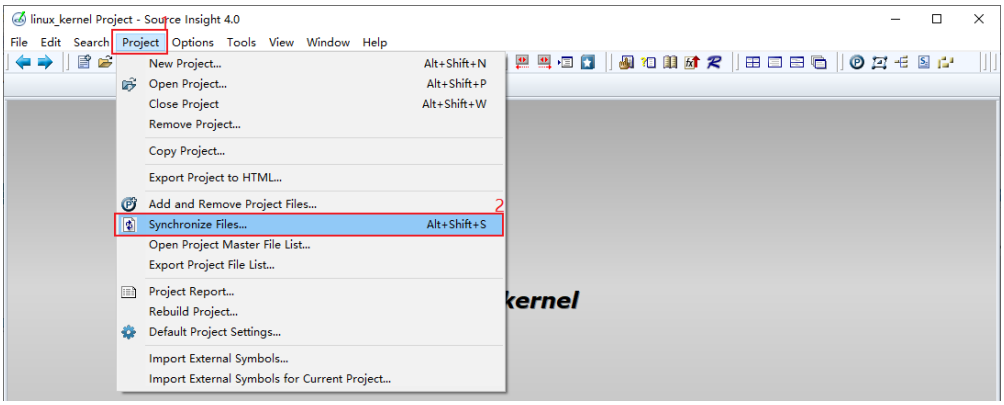


图 2.3.2.6 同步文件

7) 在弹出的 Synchronize File 对话框中 选中 **Force all files to be re-parsed** (强制解析所有文件)，这一步会生成所有代码之间的调用关系等,等待一段时间后会返回到主页面，此时可以进行阅读代码

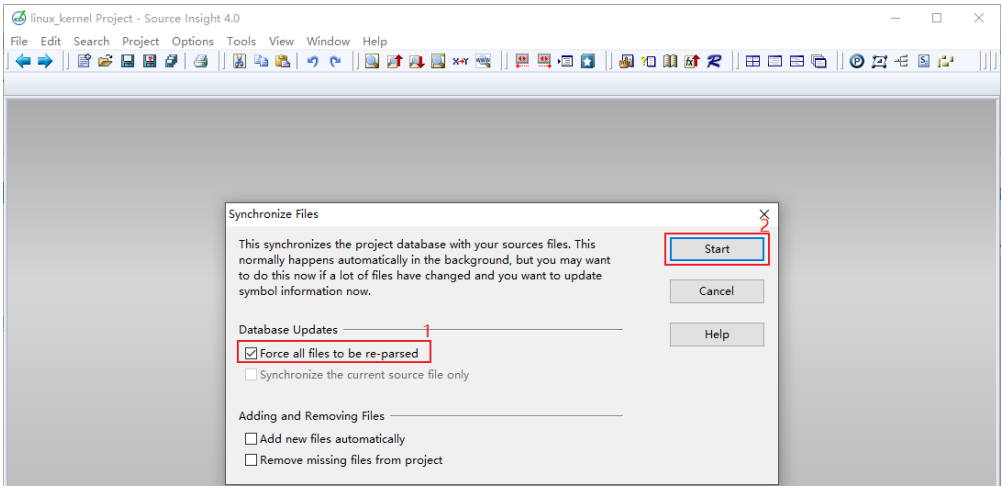


图 2.3.2.7 解析所有所有文件

2.4.3 Source Insight的操作示例

1) 点击"P"图标打开文件列表，双击文件打开文件

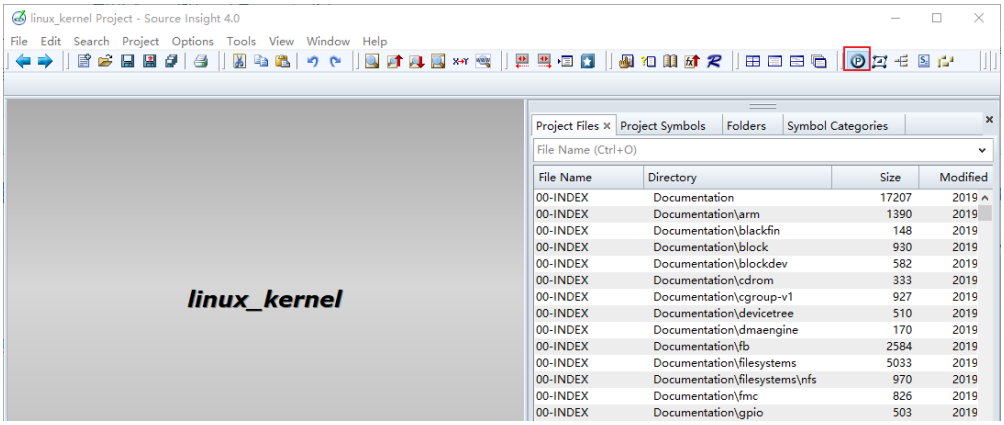


图 2.3.3.1 点击 P 图标

2) 鼠标放在函数、变量上"ctrl+鼠标点击" 跳到定义的位置。 sourceinght.gif

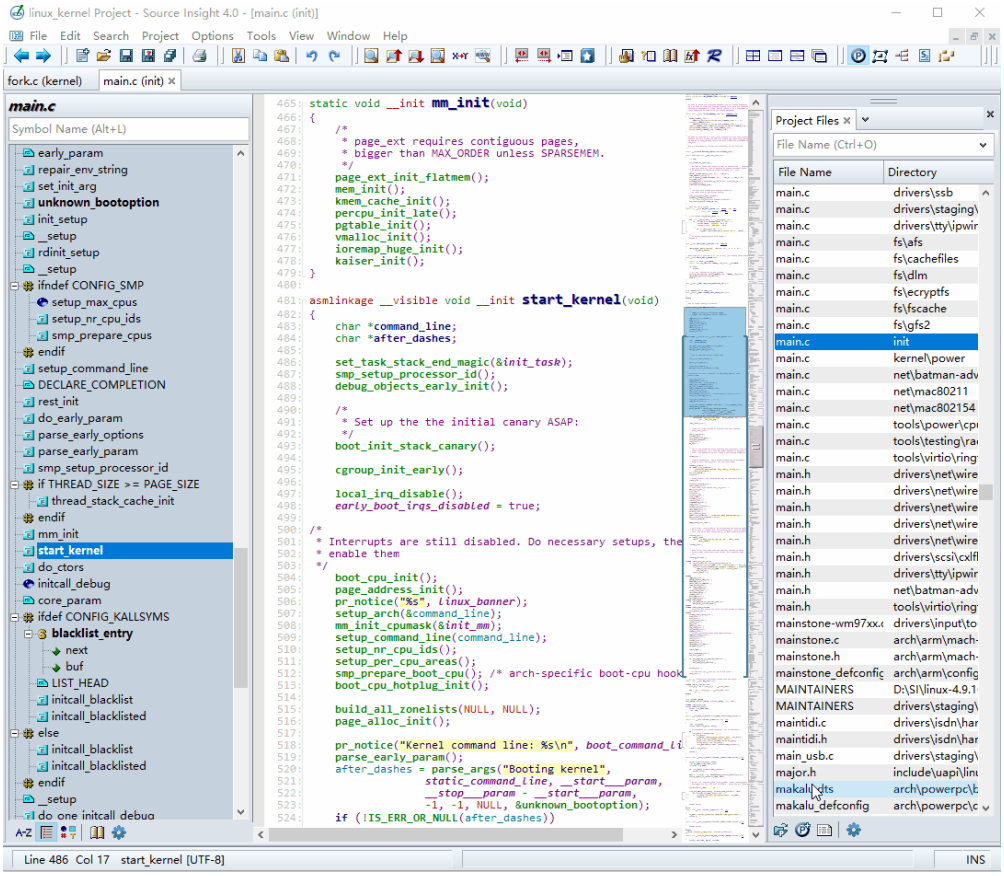


图 2.3.3.2 查找函数或变量定义

3) 双击函数, "ctrl +/"查找引用

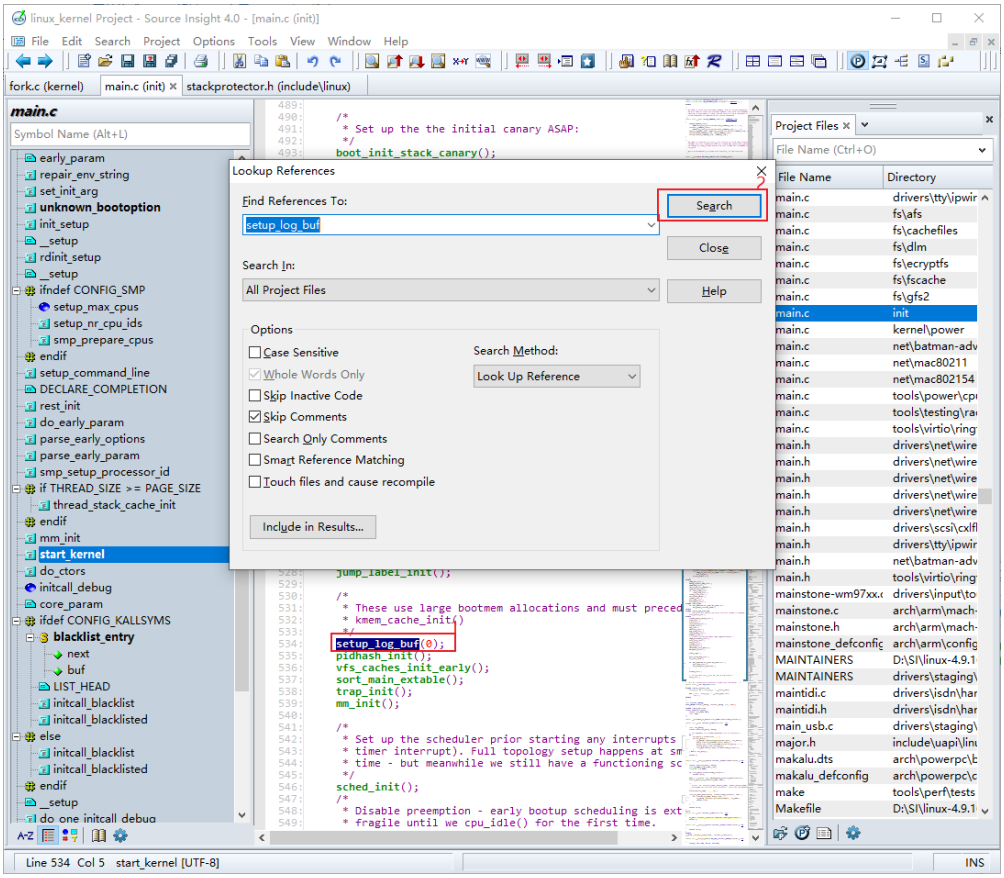


图 2.3.3.3 查找引用

其它一些常用功能键。

- Alt + , 退回上一步操作
- F5 显示行号
- F8 高亮代码或者变量

第3章　构建系统

Linux 平台上有许多开源的嵌入式 linux 系统构建框架(框架的意思就是工具)，这些框架极大的方便了开发者进行嵌入式系统的定制化构建，目前比较常见的有 openWRT, Buildroot, Yocto, Arago 等等。其中 Buildroot 功能强大，使用简单，而且采用了类似于 linux kernel 的配置和编译框架，所以受到广大嵌入式开发人员的欢迎。

本章重点介绍使用 Buildroot_2018.02LTS 版构建文件系统和 u-boot, kernel 镜像的方法，并从这三个部分入手，描述如何使用 Buildroot 构建一个适合 roc-rk3399-pc 开发板的嵌入式 Linux 系统。

3.1　获取源码

源码的获取方法有 2 种：本地拷贝、在线下载。这 2 种方法请选择 1 种，不要同时选择 2 种方法。

3.1.1　本地拷贝

通过 FileZilla 工具上传资料光盘中的(07_bsp_sdk/100ask_roc-rk3399-pc)整个文件夹到 ubuntu 系统/home/book 目录下

3.1.2　在线下载

roc-rk3399-pc 开发板的所有代码都是通过 repo 进行管理，可以通过如下命令进行下载或同步。

1) 配置 repo

下载 repo 工具前需要设置 git 的邮箱和用户名，git 邮箱和用户名请根据个人情况进行配置。

```
book@100ask:~$  git config --global user.email "you@example.com"
book@100ask:~$  git config --global user.name "Your Name"
```

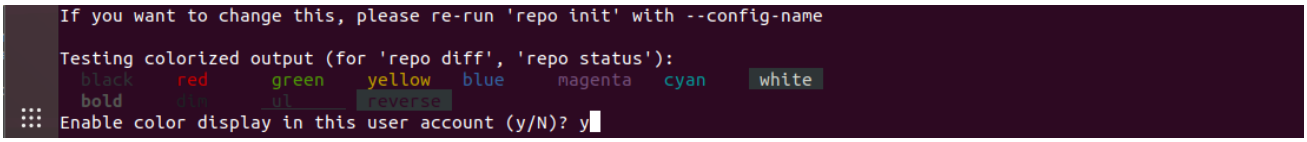
2) 下载源码

通过 repo 管理多个 git 仓库中的源码，可以及时更新最新代码，以方便开发者学习使用。

```
book@100ask:~$  git clone  https://git.dev.tencent.com/codebug8/repo.git
book@100ask:~$  mkdir -p 100ask_roc-rk3399-pc && cd 100ask_roc-rk3399-pc
book@100ask:~/100ask_roc-rk3399-pc $ ../repo/repo init -u https://dev.tencent.com/u/weidongshan/p/manifests/git -b
linux-sdk -m rk3399/roc-rk3399-pc_linux_release.xml --no-repo-verify
book@100ask:~/100ask_roc-rk3399-pc $  ../repo/repo sync -j4
```

注意：repo 在线下载的代码和资料光盘中的代码是一致的，后期方便同步更新！

首次下载如果提示 **Testing colorized output (for 'repo diff', 'repo status'):** 此时输入 **y** 即可,继续执行 ../repo/repo sync -j4 命令即可开始同步源码。



3.2　配置交叉编译工具链

~~如果你使用的不是我们提供的 ubuntu 映象文件，可以参考本节来配置交叉编译工具链。~~

3.2.1　设置交叉编译工具链

交叉编译工具链用来在 ubuntu 主机上编译应用程序，而这些应用程序是在 ARM 等其他平台上运行。

设置交叉编译工具主要是设置 PATH， ARCH 和 CROSS_COMPILE 三个环境变量，下面介绍具体设置方法。

在本文档中，源码、交叉编译工具链都是存放于/home/book 目录下；如果你的目录不一样，请自行修改本节所讲述的命令。

设置这 3 个环境变量有多种方法：

1) 永久生效

如需永久修改，请修改用户配置文件, Ubuntu 系统下，修改如下：

```
vim ~/.bashrc
```

在行尾添加或修改：

```
export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-
export PATH=$PATH:/home/book/100ask_roc-rk3399-pc/ToolChain-6.3.1/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin
```

设置完毕后，执行 source ~/.bashrc 使其生效。

2) 临时生效

执行完“export”命令后，该设置**只对当前终端有效：**

```
book@100ask:~$ export PATH=$PATH:/home/book/100ask_roc-rk3399-pc/ToolChain-6.3.1/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin
book@100ask:~$ export ARCH=arm64
book@100ask:~$ export CROSS_COMPILE=aarch64-linux-gnu-
```

3) 手动指定

先设置 PATH 环境变量，然后在 make 编译时指定 ARCH 架构 CROSS_COMPILE 交叉编译工具链：

```
book@100ask:~$ export PATH=$PATH:/home/book/100ask_roc-rk3399-pc/ToolChain-6.3.1/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin
book@100ask:~$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-
```

3.2.2 测试交叉编译工具链

测试环境变量：

```
book@100ask:~$ echo $ARCH
arm64
book@100ask:~$ echo $CROSS_COMPILE
aarch64-linux-gnu-
```

测试交叉编译器：

```
book@100ask:~$ aarch64-linux-gnu-gcc  -v
```

```
Using built-in specs.
COLLECT_GCC=aarch64-linux-gnu-gcc-6.3.1
COLLECT_LTO_WRAPPER=/home/book/100ask_roc-rk3399-pc/ToolChain-6.3.1/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/../libexec/gcc/aarch64-linux-gnu/6.3.1/lto-wrapper
Target: aarch64-linux-gnu
Configured with: '/home/tcwg-buildslave/workspace/tcwg-make-release/builder_arch/amd64/label/tcwg-x86_64-build/target/aarch64-linux-gnu/snapshots/gcc.git~linaro-6.3-2017.05/configure' SHELL=/bin/bash --with-mpc=/home/tcwg-buildslave/workspace/tcwg-make-release/builder_arch/amd64/label/tcwg-x86_64-build/target/aarch64-linux-gnu/_build/builds/destdir/x86_64-unknown-linux-gnu --with-mpfr=/home/tcwg-buildslave/workspace/tcwg-make-release/builder_arch/amd64/label/tcwg-x86_64-build/target/aarch64-linux-gnu/_build/builds/destdir/x86_64-unknown-linux-gnu --with-gmp=/home/tcwg-buildslave/workspace/tcwg-make-release/builder_arch/amd64/label/tcwg-x86_64-build/target/aarch64-linux-gnu/_build/builds/destdir/x86_64-unknown-linux-gnu --with-gnu-as --with-gnu-ld --disable-libmudflap --enable-lto --enable-shared --without-included-gettext --enable-nls --disable-sjlj-exceptions --enable-gnu-unique-object --enable-linker-build-id --disable-libstdcxx-pch --enable-c99 --enable-clocale=gnu --enable-libstdcxx-debug --enable-long-long --with-cloog=no --with-ppl=no --with-isl=no --disable-multilib --enable-fix-cortex-a53-835769 --enable-fix-cortex-a53-843419 --with-arch=armv8-a --enable-threads=posix --enable-multiarch --enable-libstdcxx-time=yes --enable-gnu-indirect-function --with-build-sysroot=/home/tcwg-buildslave/workspace/tcwg-make-release/builder_arch/amd64/label/tcwg-x86_64-build/target/aarch64-linux-gnu/_build/sysroots/aarch64-linux-gnu --with-sysroot=/home/tcwg-buildslave/workspace/tcwg-make-release/builder_arch/amd64/label/tcwg-x86_64-build/target/aarch64-linux-gnu/_build/builds/destdir/x86_64-unknown-linux-gnu/aarch64-linux-gnu/libc --enable-checking=release --disable-bootstrap --enable-languages=c,c++,fortran,lto --build=x86_64-unknown-linux-gnu --host=x86_64-unknown-linux-gnu --target=aarch64-linux-gnu --prefix=/home/tcwg-buildslave/workspace/tcwg-make-release/builder_arch/amd64/label/tcwg-x86_64-build/target/aarch64-linux-gnu/_build/builds/destdir/x86_64-unknown-linux-gnu
Thread model: posix
gcc version 6.3.1 20170404 (Linaro GCC 6.3-2017.05)
book@100ask:~$
```

3.3 解压编译 bootloader

我们使用版本为 uboot2019.01,我们提供的源码针对板子进行过修改，u-boot 官网下载的源码未测试是否可以直接使用。

3.3.1 解压u-boot源码

注意：如果你使用的是 repo 在线同步代码，则不需要下面的解压步骤。

对于本地拷贝，执行下面的命令解压 U-boot 源码：

```
book@100ask:~$ cd /home/book/100ask_roc-rk3399-pc
book@100ask:~/100ask_roc-rk3399-pc $ tar -zxvf uboot2019.01.tar.gz
```

3.3.2 编译u-boot镜像

不同的开发板对应不同的配置文件，配置文件位于 u-boot 的 configs/ 目录,下面以 Roc-rk3399-pc 开发板为例，说明 u-boot 的编译过程：

```
book@100ask:~$ cd ~/100ask_roc-rk3399-pc/uboot2019.01
book@100ask: ~/100ask_roc-rk3399-pc/uboot2019.01$ make distclean
book@100ask: ~/100ask_roc-rk3399-pc/uboot2019.01$ make 100ask_roc-rk3399-pc_defconfig
book@100ask: ~/100ask_roc-rk3399-pc/uboot2019.01$ make
book@100ask: ~/100ask_roc-rk3399-pc/uboot2019.01$ ./make.sh trust
book@100ask: ~/100ask_roc-rk3399-pc/uboot2019.01$ ./make.sh uboot
book@100ask: ~/100ask_roc-rk3399-pc/uboot2019.01$ ./make.sh loader
```

如下图 3.2.2.1 所示，生成的供 roc-rk3399-pc 启动 uboot 的镜像文件。

```
book@100ask:~/100ask_firefly-rk3288/uboot2017.09$ ls -lh trust.img  uboot.img  idbloader.img
-rw-r--r-- 1 book book  89K Jul 10 03:38 idbloader.img
-rw-r--r-- 1 book book 4.0M Jul 10 03:38 trust.img
-rw-r--r-- 1 book book 4.0M Jul 10 03:38 uboot.img
book@100ask:~/100ask_firefly-rk3288/uboot2017.09$
```

图 3.3.2.1 uboot 镜像启动文件

通过 dd 命令烧写 uboot 镜像文件，我们以 SD 卡为例，首先通过 vmware workstation 把 sd 卡设备连接到到 ubuntu 18.04 虚拟机上

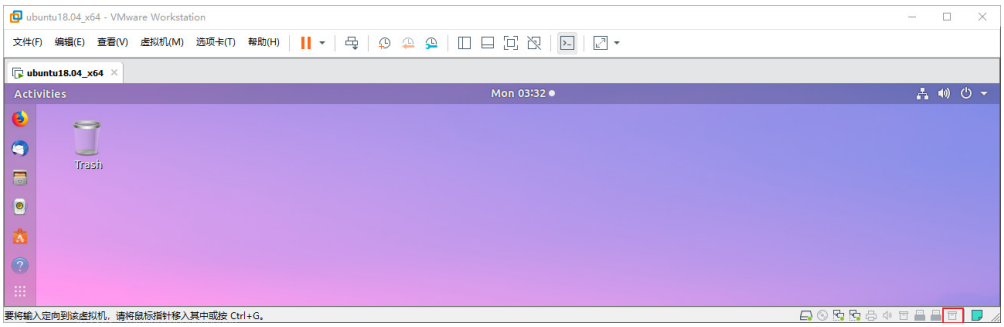


图 3.2.2.2 VMware 挂载 sd 卡设备

使用 dmesg 命令获取设备挂载的设备节点，如下图 3.2.2.3 得到的设备节点为/dev/sdb

```
book@100ask: ~/100ask_roc-rk3399-pc/uboot2019.01$ dmesg
```

```
[20862.255215] usb 1-1: new high-speed USB device number 3 using ehci-pci
[20862.636666] usb 1-1: New USB device found, idVendor=05e3, idProduct=0723, bcdDevice=94.51
[20862.636668] usb 1-1: New USB device strings: Mfr=3, Product=4, SerialNumber=2
[20862.636669] usb 1-1: Product: USB Storage
[20862.636690] usb 1-1: Manufacturer: Generic
[20862.636691] usb 1-1: SerialNumber: 000000009451
[20862.638968] usb-storage 1-1:1.0: USB Mass Storage device detected
[20862.639315] usb-storage 1-1:1.0: Quirks match for vid 05e3 pid 0723: 8000
[20862.639397] scsi host33: usb-storage 1-1:1.0
[20863.652551] scsi 33:0:0:0: Direct-Access     Generic  STORAGE DEVICE   9451 PQ: 0 ANSI: 0
[20863.653067] sd 33:0:0:0: Attached scsi generic sg2 type 0
[20863.786938] sd 33:0:0:0: [sdb] 15499264 512-byte logical blocks: (7.94 GB/7.39 GiB)
[20863.795903] sd 33:0:0:0: [sdb] Write Protect is off
[20863.795904] sd 33:0:0:0: [sdb] Mode Sense: 03 00 00 00
[20863.802863] sd 33:0:0:0: [sdb] No Caching mode page found
[20863.802865] sd 33:0:0:0: [sdb] Assuming drive cache: write through
[20863.838022]  sdb: sdb1 sdb2
[20863.862714] sd 33:0:0:0: [sdb] Attached SCSI removable disk
[20864.627165] FAT-fs (sdb2): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
[20864.760128] EXT4-fs (sdb1): recovery complete
[20864.765887] EXT4-fs (sdb1): mounted filesystem with ordered data mode. Opts: (null)
```

图 3.2.2.3　设备节点输出示意图

使用 dd 命令烧写 img 镜像文件，烧写命令如下所示:.

```
book@100ask:~/100ask_roc-rk3399-pc/uboot2019.01$   sudo dd if=idbloader.img of=/dev/sdb seek=64
book@100ask:~/100ask_roc-rk3399-pc/uboot2019.01$   sudo dd if=uboot.img of=/dev/sdb seek=16384
book@100ask:~/100ask_roc-rk3399-pc/uboot2019.01$   sudo dd if=trust.img of=/dev/sdb seek=24576
```

从 TF 卡启动，启动过程中，通过按空格键可以进入 U-Boot 命令控制台执行需要的命令。例如：

```
U-Boot 2017.09-gd80ac02 (Feb 25 2019 - 16:26:01 +0800)

Model: Firefly-RK3288
DRAM:  2 GiB
Relocation Offset is: 7de63000
MMC:   dwmmc@ff0c0000: 1, dwmmc@ff0f0000: 0
*** Warning - bad CRC, using default environment

In:    serial
Out:   serial
Err:   serial
Model: Firefly-RK3288
switch to partitions #0, OK
mmc1 is current device
do_rkimg_test found IDB in SDcard
Boot from SDcard
get part misc fail -1
Net:
Warning: ethernet@ff290000 (eth0) using random MAC address - ee:8c:3a:ae:46:60
eth0: ethernet@ff290000
Unknown command 'usb' - try 'help'
Hit any key to stop autoboot:  0
=>
```

```
=> help -- 获取帮助
=> echo -- 查看变量
```

注明： 如何设置启动参数请参考 NFSROOT 启动

3.4　编译 linux kernel

3.4.1　解压kernel源码

注意：如果你使用的是 repo 在线同步代码，则不需要下面的解压步骤。

对于本地拷贝，执行下面的命令解压 Kernel 源码：

```
book@100ask:~$ cd /home/book/100ask_roc-rk3399-pc/
book@100ask: ~/100ask_roc-rk3399-pc$ tar -zxvf linux-4.4.tar.gz
```

注意：如果你使用的是 repo 在线同步代码，则不需要上面的解压步骤

3.4.2　编译内核镜像

不同的开发板对应不同的配置文件，配置文件位内核源码 arch/arm/configs/目录。下面以 Roc-rk3399-pc 开发板为例，说明 kernel 的编译过程：

```
book@100ask:~/100ask_roc-rk3399-pc$ cd linux-4.4
```

```
book@100ask:~/100ask_roc-rk3399-pc/linux-4.4$ make mrproper
book@100ask:~/100ask_roc-rk3399-pc/linux-4.4$ make 100ask_roc-rk3399-pc_defconfig
book@100ask:~/100ask_roc-rk3399-pc/linux-4.4$ make Image
book@100ask:~/100ask_roc-rk3399-pc/linux-4.4$ make dtbs
```

编译完成后，在 arch/arm/boot 目录下生成 Image 文件，在 arch/arm/boot/dts 目录下生成设备树的二进制 rk3288-firefly.dtb 文件。

3.4.3 编译内核模块并安装

编译内核模块

```
book@100ask:~/100ask_roc-rk3399-pc/linux-4.4$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- modules
```

安装内核模块到 nfs 根文件系统

```
book@100ask:~/100ask_roc-rk3399-pc/linux-4.4$ sudo make ARCH=arm64 INSTALL_MOD_PATH=/media/rootfs
modules_install
```

3.4.4 常见问题

- 常见问题 1，编译时只设置了交叉编译工具链，没有设置 ARCH，提示的报错信息！

```
  UPD     include/generated/uapi/linux/version.h
  CHK     include/generated/utsrelease.h
  UPD     include/generated/utsrelease.h
  CC      kernel/bounds.s
gcc: error: unrecognized command line option '-mlittle-endian'; did you mean '-fconvert=little-endi
'?
Kbuild:19: recipe for target 'kernel/bounds.s' failed
make[1]: *** [kernel/bounds.s] Error 1
Makefile:1074: recipe for target 'prepare0' failed
make: *** [prepare0] Error 2
book@100ask:~/100ask_roc-rk3399-pc/linux-4.4$
```

解决办法:编译时指定 ARCH=arm64 或者参考设置交叉编译工具链，在 ~/.bashrc 设置为永久生效。

- 常见问题 2，编译内核时提示 scripts/extract-cert.c:21:10 fatal error: openssl/bip.h: No such file or directory 此错误。

```
scripts/extract-cert.c:21:10: fatal error: openssl/bio.h: No such file or directory
 #include <openssl/bio.h>
          ^~~~~~~~~~~~~~~
compilation terminated.
scripts/Makefile.host:91: recipe for target 'scripts/extract-cert' failed
make[1]: *** [scripts/extract-cert] Error 1
Makefile:584: recipe for target 'scripts' failed
```

解决办法: 在终端下执行 sudo apt-get install libssl-dev 安装此软件包即可。

3.5 构建根文件系统

3.5.1 解压buildroot源码

注意：如果你使用的是 repo 在线同步代码，则不需要下面的解压步骤。

对于本地拷贝，执行下面的命令解压 buildroot 源码：

```
book@100ask:~$ cd /home/book/100ask_roc-rk3399-pc
book@100ask:~/100ask_roc-rk3399-pc$ tar -zxvf buildroot2018.02.tar.gz
```

3.5.2 配置文件说明

配置文件	含义
100ask_roc-rk3399-pc_defconfig	默认文件系统版本

3.5.3 编译系统

下面以 100ask_roc-rk3399-pc_defconfig 配置文件为例，说明 buildroot 的配置编译过程：

```
book@100ask:~/100ask_roc-rk3399-pc$ cd buildroot2018.02
book@100ask:~/100ask_roc-rk3399-pc/buildroot2018.02$ make clean
```

```
book@100ask:~/100ask_roc-rk3399-pc/buildroot2018.02$ make 100ask_roc-rk3399-pc_defconfig
book@100ask:~/100ask_roc-rk3399-pc/buildroot2018.02$ make menuconfig
book@100ask:~/100ask_roc-rk3399-pc/buildroot2018.02$ make –jN              /* N表示CPU的线程数，比如可以写4、8等以加快编译速度 */
```

注意：机器性能不同，编译时间不同。性能差的电脑，有可能需要等待 1 ~ 2 个小时。

3.5.4 镜像文件

编译成功后文件输出路径为 **output/images**

```
buildroot2018.02
        ├── output
            ├── images
                ├── idbloader.img                <--u-boot镜像
                ├── nfs_rootfs                   <--打包并压缩的根文件系统，适用于NFSROOT启动
                │   └── rootfs.tar.gz
                ├── rk3288-firefly.dtb           <--设备树文件
                ├── rootfs.ext2
                ├── rootfs.ext4 -> rootfs.ext2   <--ext2格式根文件系统
                ├── rootfs.tar                   <--打包后的根文件系统
                ├── sdcard.img                   <--完整的SD卡系统镜像
                ├── trust.img                    <--u-boot镜像
                ├── u-boot.bin
                ├── u-boot-dtb.img
                ├── uboot.img                    <--u-boot镜像
                └── zImage                       <--内核镜像
```

注明：如何通过 NFS 系统根文件系统进行调试，请参考 NFSROOT 启动章节

第4章 烧写系统

本章主要介绍 roc-rk3399-pc 的两种启动方式，分别，TF 卡启动，nfs 网络挂载方式启动，用户可以根据自己的需要选择适合自己的进行安装配置。

本章需要使用的软件：

- SD Card Formatter 5.0.1 Setup.exe
- win32diskimager-1.0.0-install.exe

Roc-rk3399-pc 开发板支持 SD 卡、emmc, nfs 网络挂载启动方式，目前我们并未提供 emmc 启动方式，如需使用 emmc 启动请参考 firefly 官方文档。

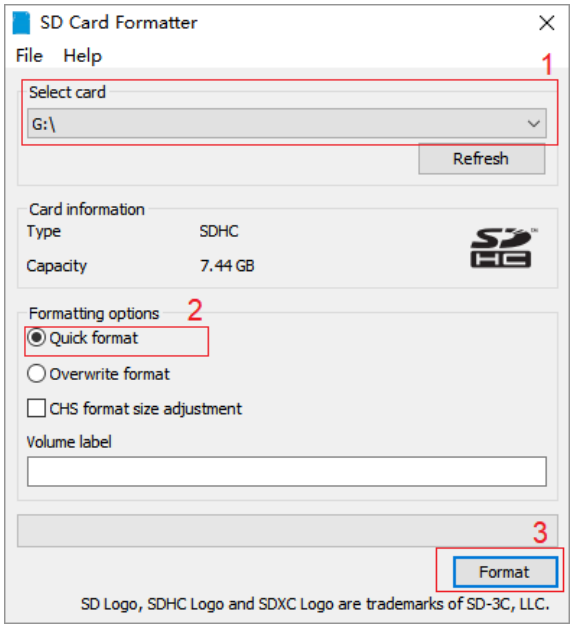
4.1 SD 卡启动

4.1.1 安装映象烧写工具win32diskimager

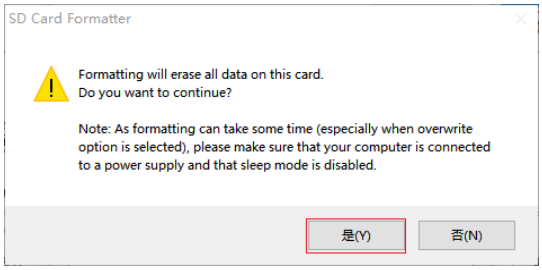
安装资料光盘下 **01_tools/ win32diskimager-1.0.0-install.exe** 软件，以后运行它时要“以管理员身份运行”。

4.1.2 格式化sd卡

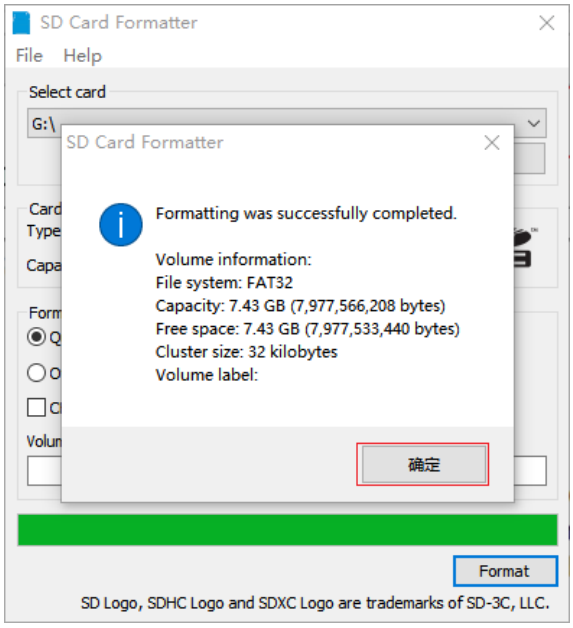
1. 安装资料光盘下 01_tools/SD Card Formatter 5.0.1 Setup.exe 软件，安装完成后打开！
2. 使用 SdCardFormatter 格式化 SD 卡，格式化步骤如下如所示!



(1)选择要格式化的 SD 卡，选中 **Quick format**，点击 **Format**



(2)在弹出的对话框中点击**是(Y)**



(3)等待格式化完成，在弹出的对话框中点击**确定**

4.1.3 烧写映像文件

1) 把 SD 卡通过读卡器插入电脑
2) 使用 win32diskimager 烧写映像文件 sdcard.img

“以管理员身份运行”win32diskimager，如下图选择 SD 卡、选择映像文件 sdcard.img，然后点击“写入”,操作步骤如下图 4.2.2.1 所示：

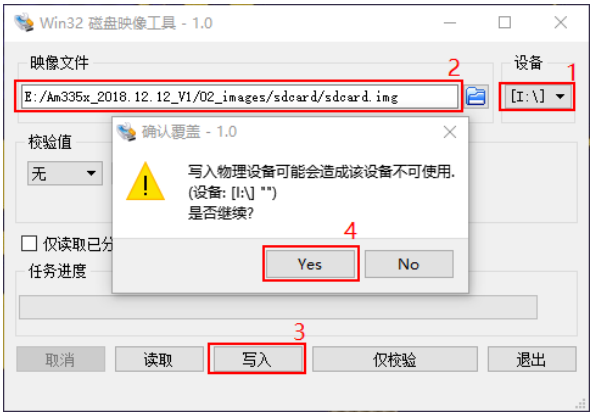


图 4.1.3.1 镜像文件烧写示意图

上图中各序号含义为：

1. 选择 SD 卡
2. 选择映象文件 sdcard.img
3. 点击“写入”
4. 确定要写入。

烧写成功后，弹出对话框“**写入成功**”，此时点击 OK，拔下 SD 卡，将启动开关拨到 SD 卡处，上电启动开发板。

4.2 NFSROOT 启动

Buildroot 编译完成之后生成的 rootfs.tar.gz，可以解压之后放到 NFS 服务器上作为 NFS ROOT 文件系统供开发板使用。使用 NFS 文件系统，便于程序的开发调试。所谓 NFS 服务器，就是我们在 VMWare 上运行的 Ubuntu。

使用 NFS 文件系统时，我们一般还会在 u-boot 使用 tftpboot 命令从 ubuntu 中下载内核文件 zImage，所以 ubuntu 上既要配置 NFS 服务，也要配置 TFTP 服务。

4.2.1 安装TFTP服务端：

在 ubuntu 中执行以下命令安装 TFTP 服务：

```
book@100ask:~$ sudo apt-get install tftp-hpa tftpd-hpa
```

然后，创建 TFTP 服务器工作目录,并打开 TFTP 服务配置文件,如下:

```
book@100ask:~$ mkdir -p /home/book/tftpboot
book@100ask:~$ chmod 777 /home/book/tftpboot
book@100ask:~$ sudo vim /etc/default/tftpd-hpa
```

在配置文件/etc/default/tftpd-hpa 中，添加以下字段：

```
TFTP_DIRECTORY="/home/book/tftpboot"
TFTP_OPTIONS="-l -c -s"
```

最后，重启 TFTP 服务:

```
book@100ask:~$ sudo service tftpd-hpa restart
```

4.2.2 安装NFS服务：

NFS 即网络文件系统，允许开发板直接通过网络挂载 PC 机中的文件夹。下面介绍在 ubuntu 上的 NFS 服务安装和配置。

首先，执行以下命令安装 NFS 服务：

```
book@100ask:~$ sudo apt-get install nfs-kernel-server
```

然后编辑/etc/exports 文件，添加 NFS 服务导出的工作目录：

```
book@100ask:~$ sudo vim /etc/exports
```

添加 NFS 目录：下面以/home/book/nfs_rootfs 为例，将其添加到/etc/exports 文件中，如下：

```
/home/book/nfs_rootfs    *(rw,nohide,insecure,no_subtree_check,async,no_root_squash)
```

最后，重启 NFS 服务:

```
book@100ask:~$ sudo service nfs-kernel-server restart
```

如果一切正常，可以在 ubuntu 中测试 NFS 服务：

```
book@100ask:~$ sudo    mount    -t    nfs    127.0.0.1:/home/book/nfs_rootfs       /mnt
```

在开发板上，可以运行这样的命令挂载NFS：mount -t nfs -o nolock,vers=3 xxx.xxx.xxx.xxx:/home/book/nfs_rootfs /mnt
假设Ubuntu的IP是xxx.xxx.xxx.xxx

4.2.3 在ubuntu中准备好文件

可以通过 tftpboot 命令从 ubuntu 中下载内核文件 zImage rk3288-firefly.dtb，可以设置内核的启动参数

让单板启动后通过 NFS 挂载 ubuntu 中的某个目录作为根文件系统。

注意：所谓根文件系统就是类似 Windows 的 C 盘，里面存放有必须的 APP、库文件、配置文件。通过 NFS 可以把 Ubuntu 的某个目录，当作板子的“C 盘”——Linux 中称之为根文件系统。

在操作单板之前，需要在 ubuntu 上准备好这些内容：zImage rk3288-firefly.dtb、nfs_rootfs 文件夹。

1) 拷贝内核和设备树文件到 tftp 目录：

将出厂镜像或者自行编译的 zImage 和设备树文件 rk3288-firefly.dtb，拷贝到 ubuntu 的 /home/book/tftpboot 目录。

2) 把根文件系统压缩包解压到 NFS 目录：

把使用 buildroot 构建得到的根文件系统 nfs_rootfs/rootfs.tar.gz，复制、解压到 ubuntu 的/etc/exports 文件中指定的目录里：

```
book@100ask:~/100ask_roc-rk3399-pc/buildroot2018.02/output/images$ cp -rf nfs_rootfs/   ~
book@100ask:~/100ask_roc-rk3399-pc/buildroot2018.02/output/images$ cd ~/nfs_rootfs
book@100ask:~/nfs_rootfs$ sudo tar -zxvf rootfs.tar.gz
```

4.2.4 在u-boot中通过网络启动单板

既然要使用网络启动单板，那么双方(开发板、ubuntu)就要确保网络是联通的，假设 ubuntu 的 IP 为 192.168.1.132，开发板的 IP 为 192.168.1.112。

首先，在 U-boot 中设置开发板 IP 为 192.168.1.112，如下:

```
=> setenv ipaddr 192.168.1.112
```

然后，在 U-boot 中使用 ping 命令测试开发板与 NFS 服务器是否连通(出现“alive”就表示联通):

```
=> ping 192.168.1.111
```

最后，在 U-Boot 控制台以下命令启动单板:

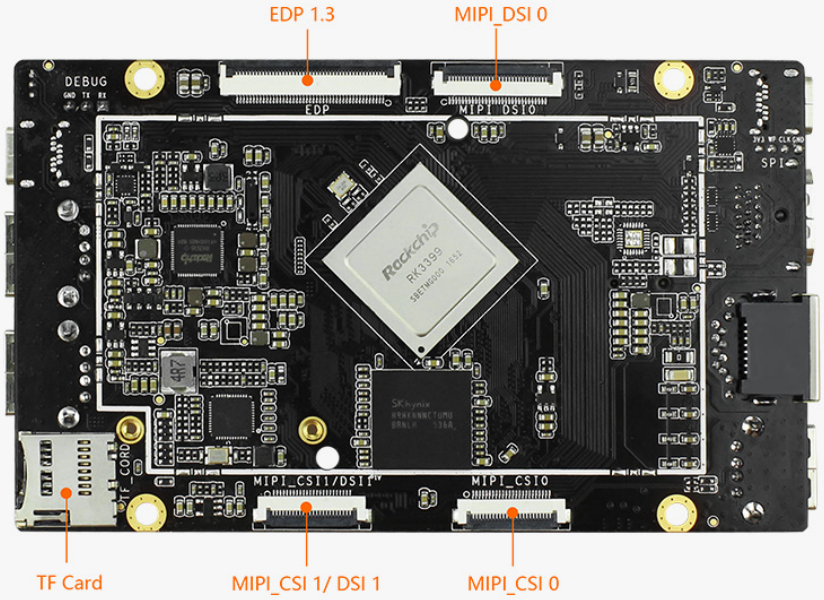
```
=> setenv serverip 192.168.1.132
=> setenv ipaddr 192.168.1.112
=> tftpboot 0x01f00000 rk3399-roc-pc.dtb; tftpboot 0x02080000 Image; setenv bootargs root=/dev/nfs
nfsroot=192.168.1.132:/home/book/rootfs,vers=3    rw ip=192.168.1.120; booti 0x02080000 - 0x01f00000
```

等待网络开始下载相应的文件并启动内核，挂载文件系统

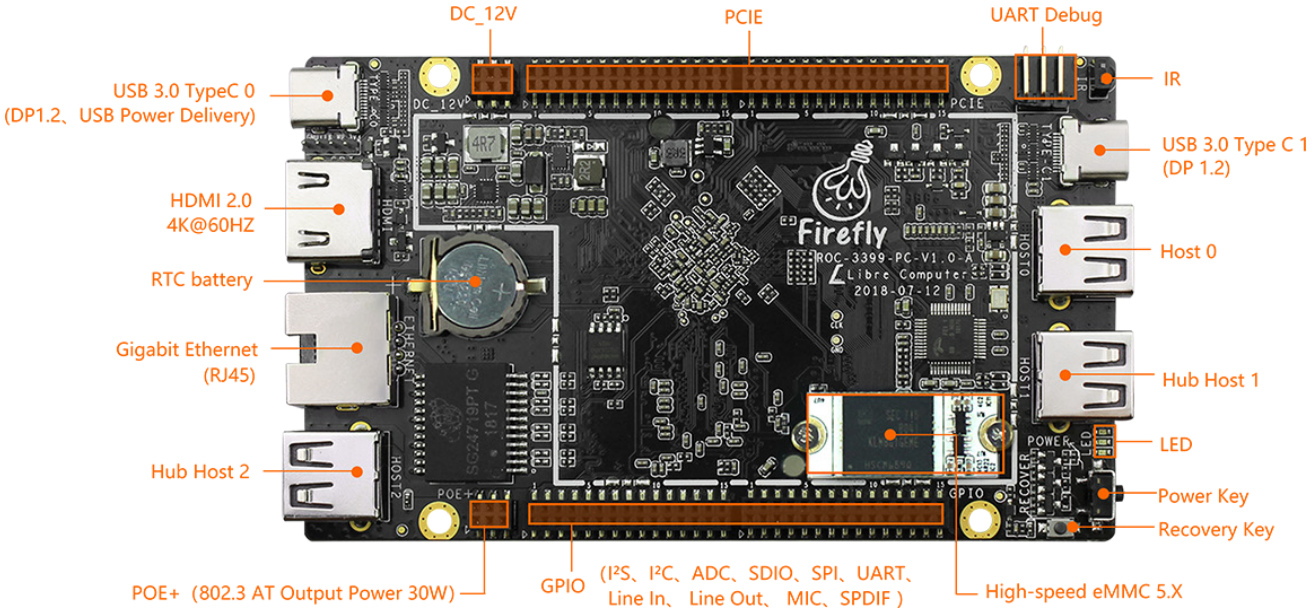
第5章平台功能测试

本章介绍如何测试板载功能，开发板各个功能部件所在位置以及名称如下图所示。本章所有的功能测试均在串口登录终端下进行。

背面视图



正面视图



5.1 网卡接口测试

此节演示在串口终端下如何设置开发板的 ip 地址，测试网络的连通性。并使用电脑通过网络浏览器访问开发板，验证开发板的 http+php 可用,如图 5.1.1.1 所示，将网线插到此接口处。

注意：既然是在开发板和电脑之间测试网如，那双方需要有网络连接。两者之间需要有一个路由器，开发板通过网线与路由器连接。而电脑与路由器之间，可以使用网线连接，也可以使用 WIFI 连接。

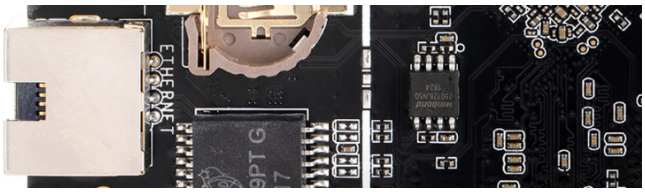


图 5.1.1 网卡接口位置示意图

1) 通过 udhcpc 命令获取 ip 地址，详细信息如图 5.1.1 所示：

```
[root@roc-rk3399-pc:~]# udhcpc
udhcpc: started, v1.27.2
udhcpc: sending discover
udhcpc: sending select for 192.168.1.90
udhcpc: lease of 192.168.1.90 obtained, lease time 172800
deleting routers
adding dns 192.168.1.1
adding dns 192.168.1.1
[root@roc-rk3399-pc:~]#
```

图 5.1.2 udhcpc 获取 IP 输出示意图

通过上图可知，使用 udhcpc 命令使开发板获得 IP 地址 192.168.1.52(你的开发板自动获取的 IP 可能不一样)。

如果通过 udhcpc 命令无法获得 IP，也可以使用 ifconfig 命令强制设置 IP：

```
[root@roc-rk3399-pc:~]# ifconfig eth0 192.168.1.112
```

2) 通过 ifconfig 命令查看详细 ip 信息：

```
[root@roc-rk3399-pc:~]# ifconfig
eth0      Link encap:Ethernet  HWaddr FA:3E:77:B9:CE:D4
          inet addr:192.168.1.90  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::66e:aac:2bed:ab27/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:644 errors:0 dropped:0 overruns:0 frame:0
          TX packets:175 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:47579 (46.4 KiB)  TX bytes:24422 (23.8 KiB)
          Interrupt:24
```

3) 网络连通性测试：

在开发板上执行如下命令，如果有数据返回则表示开发板跟互联网是连通的(前提是你的路由器是可以上网的)：

```
[root@roc-rk3399-pc:~]# ping www.baidu.com
```

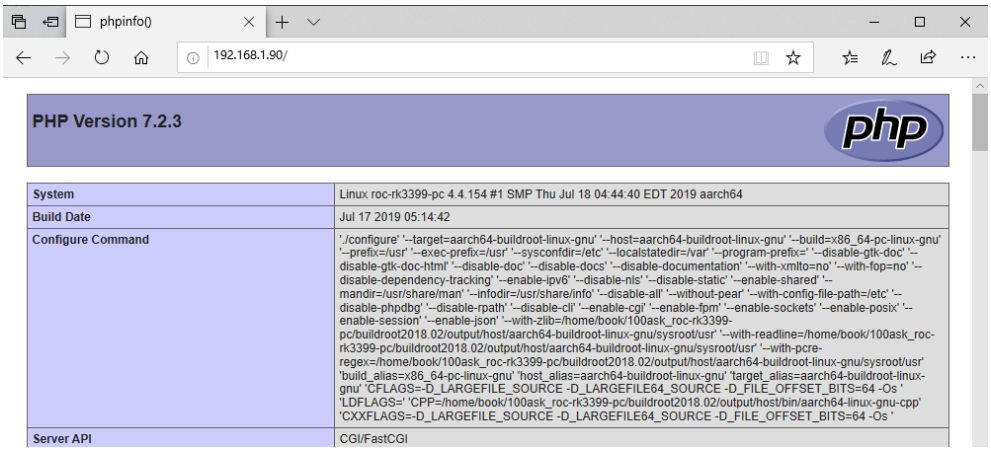
```
[root@roc-rk3399-pc:~]# ping www.baidu.com
PING www.a.shifen.com (14.215.177.38) 56(84) bytes of data.
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=1 ttl=56 time=7.53 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=2 ttl=56 time=7.32 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=3 ttl=56 time=15.1 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=4 ttl=56 time=7.38 ms
^C
--- www.a.shifen.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 7.327/9.341/15.119/3.338 ms
[root@roc-rk3399-pc:~]#
```

当然，很多时候开发板不能正通互联网，这也没关系，只要能 ping 通我们的 Windows、Ubuntu 即可(例如我们的 ubuntu IP 地址是 192.168.1.163)如下为 ping 通示例：

```
[root@roc-rk3399-pc:~]# ping 192.168.1.163
PING 192.168.1.163 (192.168.1.163) 56(84) bytes of data.
64 bytes from 192.168.1.163: icmp_seq=1 ttl=64 time=1.15 ms
64 bytes from 192.168.1.163: icmp_seq=2 ttl=64 time=0.869 ms
64 bytes from 192.168.1.163: icmp_seq=3 ttl=64 time=1.79 ms
64 bytes from 192.168.1.163: icmp_seq=4 ttl=64 time=0.664 ms
^C
--- 192.168.1.163 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.664/1.121/1.794/0.426 ms
[root@roc-rk3399-pc:~]#
```

4) http+Php 测试：

假设开发板的 IP 为 **192.168.1.52**，在电脑的网络浏览器中输入此 IP，就能看到此开发板所支持的 php 特征描述：



5.2 USB Host 接口测试

此节演示在终端下如何在 USB Host 接口上使用 usb 存储设备，将 usb 设备插图图 5.2.1 所示的 usbhost 接口处。

注意：需要准备一个 U 盘、USB 蓝牙模块、usb 网卡或者 usb 摄像头等 USB 设备。

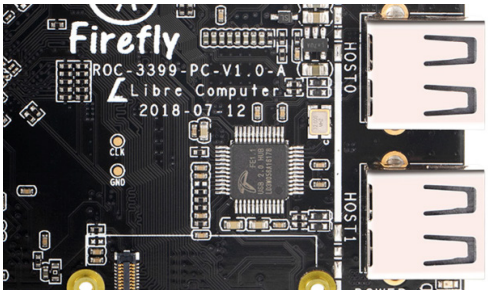


图 5.2.1USB host 接口位置示意图

下面使用一个 16G 的 U 盘作为例子，插到任意一个 USB Host 接口，会打印出如下设备信息：

```
[root@roc-rk3399-pc:~]# [  701.500894] usb 2-1.4: new high-speed USB device number 3 using ehci-platform
[  701.601980] usb 2-1.4: New USB device found, idVendor=13fe, idProduct=5500
[  701.622250] usb 2-1.4: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[  701.642931] usb 2-1.4: Product: V60
[  701.659340] usb 2-1.4: Manufacturer: EAGET
[  701.676327] usb 2-1.4: SerialNumber: 070D5CA1380C8F92
[  701.696621] usb-storage 2-1.4:1.0: USB Mass Storage device detected
[  701.708756] scsi host0: usb-storage 2-1.4:1.0
[  702.725258] scsi 0:0:0:0: Direct-Access     EAGET    V60              PMAP PQ: 0 ANSI: 6
[  702.744368] sd 0:0:0:0: [sda] 30302208 512-byte logical blocks: (15.5 GB/14.4 GiB)
[  702.767735] sd 0:0:0:0: [sda] Write Protect is off
[  702.778991] sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
[  702.965365]  sda: sda1 sda2
[  702.984580] sd 0:0:0:0: [sda] Attached SCSI removable disk
[  703.276217] FAT-fs (sda2): utf8 is not a recommended IO charset for FAT filesystems, filesystem will be case sensitive!
[  703.312150] FAT-fs (sda2): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.

[root@roc-rk3399-pc:~]#
```

通过打印的设备信息可知，系统为该 usb 存储设备创建的设备节点为 /dev/sda。一般来说/dev/sda 对应整个 U 盘，/dev/sda1 对应该 U 盘的第 1 个分区，/dev/sda2 对应第 2 个分区。

有些 U 盘没有划分分区，它只有一个设备节点/dev/sda，而没有/dev/sda1 等节点。对于这种情况，/dev/sda 即代表整个 U 盘，也代表第 1 个分区。

我们可以挂载某个分区，挂载之前要先通过 fdisk 命令获取分区类型,如下所示：

```
[root@roc-rk3399-pc:~]# fdisk  -l  /dev/sda
[root@roc-rk3399-pc:~]# fdisk -l /dev/sda
Disk /dev/sda: 14 GB, 15514730496 bytes, 30302208 sectors
1886 cylinders, 255 heads, 63 sectors/track
Units: cylinders of 16065 * 512 = 8225280 bytes

Device  Boot StartCHS    EndCHS        StartLBA     EndLBA    Sectors  Size Id Type
/dev/sda1 *  38,62,25    1023,254,63      614400   29687807   29073408 13.8G  7 HPFS/NTFS
/dev/sda2    1023,254,63 1023,254,63   29687808   30298110     610303  297M  6 FAT16
[root@roc-rk3399-pc:~]#
```

从上图可知/dev/sda2 是 FAT16 格式，挂载是可以指定类型为“vfat”：

```
[root@roc-rk3399-pc:~]# mount   -t vfat /dev/sda2 /mnt
```

```
[root@roc-rk3399-pc:~]# mount  -t vfat /dev/sda2 /mnt
[root@roc-rk3399-pc:~]# ls /mnt/
EFI                     System Volume Information  WEPE
[root@roc-rk3399-pc:~]#
```

测试完以后，通过 umount 卸载/mnt，才可拔下 usb 设备：

```
[root@roc-rk3399-pc:~]# umount   /mnt
```

5.3 OTG 接口测试

此节演示如何测试 OTG 接口的两种模式，分别是 device 模式和 host 模式。

注意：需要准备一个 OTG 转接头(开发板清单中不配)、type C usb 数据线(开发板清单里配有)。这 2 种设备如下图所示：

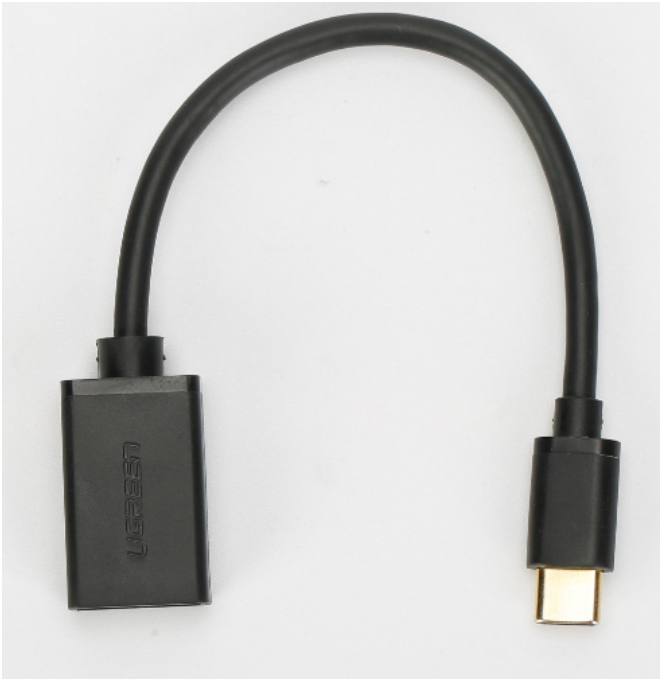


图 5.3.1 OTG 转换头



图 5.3.2 type c usb 线

1) otg device 模式测试：

使用 type usb 线小的一头插入如下图 5.3.1 所示的红色框所在接口，另一端连接电脑 USB 接口，typeC otg 接口位置如下图 5.3.1 所示：

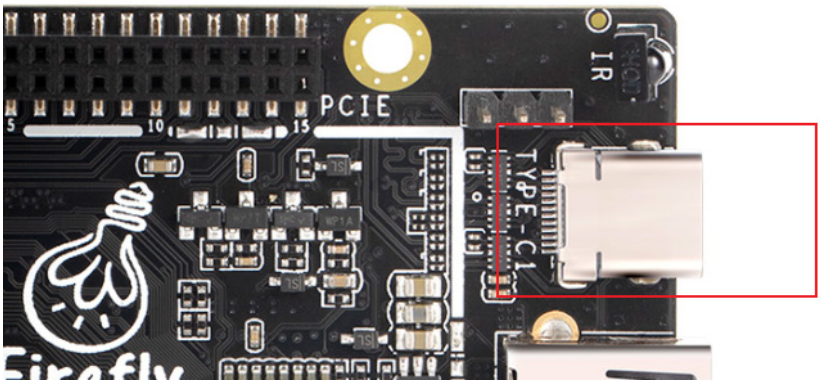


图 5.3.1 otg device 模式连接示意图

开发板作为 USB 从设备，接到电脑上；电脑会把开发板识别为一个 U 盘。

先执行以下命令卸载正在运行的驱动程序：

```
[root@roc-rk3399-pc:~]# rmmod g_mass_storage
```

```
[root@roc-rk3399-pc:~]# modprobe  g_mass_storage file=/dev/mmcblk0p2   removable=1
[ 1090.334580] Mass Storage Function, version: 2009/09/11
[ 1090.352044] LUN: removable file: (no medium)
[ 1090.369255] LUN: removable file: /dev/mmcblk0p2
[ 1090.385785] Number of LUNs=1
[ 1090.401771] g_mass_storage gadget: Mass Storage Gadget, version: 2009/09/11
[ 1090.413669] g_mass_storage gadget: userspace failed to provide iSerialNumber
[ 1090.425619] g_mass_storage gadget: g_mass_storage ready
[root@roc-rk3399-pc:~]# [ 1103.535108] phy phy-ff770000.syscon:usb2-phy@e460.5: charger = USB_CDP_CHARGER
[ 1103.683285] fusb302 4-0022: CC connected in CC1 as UFP
[ 1103.697656] cdn-dp fec00000.dp: [drm:cdn_dp_pd_event_work] Not connected. Disabling cdn
[ 1103.711854] rockchip-dwc3 usb1: USB peripheral connected
[ 1106.648336] fusb302 4-0022: PD disabled
[ 1134.179656] g_mass_storage gadget: high-speed config #1: Linux File-Backed Storage
```

此时在电脑的设备管理器中可以看到一个名为 linux File-Stor Gadget USB Device 的磁盘设备，在 windows 资源管理器中也可以看到有新的可移动设备盘符弹出：



测试完成后，在终端下执行“rmmod g_mass_storage ”卸载改模块驱动：

```
[root@roc-rk3399-pc:~]# rmmod g_mass_storage
```

2) otg host 模式：

开发板作为 usb 主设备，其他 USB 设备通过 otg 转接头插入开发板，开发板即可识别出这些 USB 外设备。

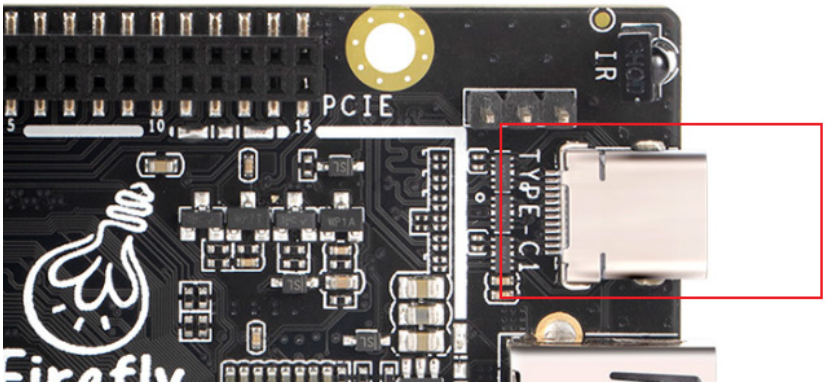


图 5.3.2.1 otg 转接头连接示意图

下图是把 U 盘通过 otg 转接头插入开发板后，在串口打印的信息：

```
[root@roc-rk3399-pc:~]# [  881.581682] usb 2-1.4: USB disconnect, device number 3
[  889.803611] fusb302 4-0022: CC connected in CC1 as DFP
[  889.828808] cdn-dp fec00000.dp: [drm:cdn_dp_pd_event_work] Not connected. Disabling cdn
[  889.842832] xhci-hcd xhci-hcd.9.auto: xHCI Host Controller
[  889.855026] xhci-hcd xhci-hcd.9.auto: new USB bus registered, assigned bus number 5
[  889.868941] xhci-hcd xhci-hcd.9.auto: hcc params 0x0220fe64 hci version 0x110 quirks 0x02030010
[  889.880668] xhci-hcd xhci-hcd.9.auto: irq 227, io mem 0xfe900000
[  889.889788] usb usb5: New USB device found, idVendor=1d6b, idProduct=0002
[  889.898982] usb usb5: New USB device strings: Mfr=3, Product=2, SerialNumber=1
[  889.908557] usb usb5: Product: xHCI Host Controller
[  889.915720] usb usb5: Manufacturer: Linux 4.4.154 xhci-hcd
[  889.923126] usb usb5: SerialNumber: xhci-hcd.9.auto
[  889.930641] hub 5-0:1.0: USB hub found
[  889.936359] hub 5-0:1.0: 1 port detected
[  889.942187] xhci-hcd xhci-hcd.9.auto: xHCI Host Controller
[  889.949836] xhci-hcd xhci-hcd.9.auto: new USB bus registered, assigned bus number 6
[  889.959225] usb usb6: We don't know the algorithms for LPM for this host, disabling LPM.
[  889.968831] usb usb6: New USB device found, idVendor=1d6b, idProduct=0003
[  889.977090] usb usb6: New USB device strings: Mfr=3, Product=2, SerialNumber=1
[  889.985813] usb usb6: Product: xHCI Host Controller
[  889.992033] usb usb6: Manufacturer: Linux 4.4.154 xhci-hcd
[  889.998869] usb usb6: SerialNumber: xhci-hcd.9.auto
[  890.005459] hub 6-0:1.0: USB hub found
[  890.010595] hub 6-0:1.0: 1 port detected
[  890.016045] rockchip-dwc3 usb1: USB HOST connected
[  890.243693] usb 5-1: new high-speed USB device number 2 using xhci-hcd
[  890.378738] usb 5-1: New USB device found, idVendor=13fe, idProduct=5500
[  890.398010] usb 5-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[  890.417782] usb 5-1: Product: V60
[  890.433579] usb 5-1: Manufacturer: EAGET
[  890.450014] usb 5-1: SerialNumber: 070D5CA1380C8F92
[  890.469685] usb-storage 5-1:1.0: USB Mass Storage device detected
[  890.481438] scsi host1: usb-storage 5-1:1.0
[  891.495788] scsi 1:0:0:0: Direct-Access     EAGET    V60              PMAP PQ: 0 ANSI: 6
[  891.521953] sd 1:0:0:0: [sdb] 30302208 512-byte logical blocks: (15.5 GB/14.4 GiB)
[  891.536883] sd 1:0:0:0: [sdb] Write Protect is off
[  891.551223] sd 1:0:0:0: [sdb] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
[  891.736389]  sdb: sdb1 sdb2
[  891.760906] sd 1:0:0:0: [sdb] Attached SCSI removable disk
[  895.723610] fusb302 4-0022: PD disabled
[  897.070687] FAT-fs (sdb2): utf8 is not a recommended IO charset for FAT filesystems, filesystem will be case sensitive!
[  897.102980] FAT-fs (sdb2): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
```

通过打印的设备信息可知，系统为该 usb 存储设备创建的设备节点为 /dev/sda。一般来说/dev/sda 对应整个 U 盘，/dev/sda1 对应该 U 盘的第 1 个分区，/dev/sda2 对应第 2 个分区。

有些 U 盘没有划分分区，它只有一个设备节点/dev/sda，而没有/dev/sda1 等节点。对于这种情况，/dev/sda 即代表整个 U 盘，也代表第 1 个分区。

我们可以挂载某个分区，挂载之前要先通过 fdisk 命令获取分区类型,如下所示：

```
[root@roc-rk3399-pc:~]# fdisk -l /dev/sda
Disk /dev/sda: 14 GB, 15514730496 bytes, 30302208 sectors
1886 cylinders, 255 heads, 63 sectors/track
Units: cylinders of 16065 * 512 = 8225280 bytes

Device  Boot StartCHS    EndCHS        StartLBA     EndLBA    Sectors  Size Id Type
/dev/sda1 *  38,62,25    1023,254,63     614400   29687807   29073408 13.8G  7 HPFS/NTFS
/dev/sda2    1023,254,63 1023,254,63   29687808   30298110     610303  297M  6 FAT16
[root@roc-rk3399-pc:~]#
```

从上图可知/dev/sda2 是 FAT16 格式，挂载是可以指定类型为“vfat”：

```
[root@roc-rk3399-pc:~]# mount  -t vfat /dev/sda2 /mnt
[root@roc-rk3399-pc:~]# ls /mnt/
EFI                     System Volume Information  WEPE
[root@roc-rk3399-pc:~]#
```

测试完以后，通过 umount 卸载/mnt，才可拔下 usb 设备：

```
[root@Roc-rk3399-pc:~]# umount   /mnt
```

5.4 RTC 测试

此节演示通过使用 date 和 hwclock 命令设置系统时间、硬件时间，并测试当操作系统重启后，系统时钟与硬件时间是否同步。

一般的板子都会有一个名为 RTC(实时时钟)的硬件，RTC 使用钮扣电池供电，在系统关闭时用来维持时钟。RTC 维持的时间，被称为硬件时间。

Linux 系统启动之后，它会自己维持时间，这个时间被称为系统时间。系统时间的初始值来源有二：

① 如果没有 RTC，系统时间初始值为 1970 年 1 月 1 日 0 点 0 分 0 秒
② 如果有 RTC，Linux 启动后，系统时间初始值从 RTC 读取

在实际使用过程中，要注意系统时间、硬件时间的同步问题：
① 使用 date 命令查看、设置系统时间，在设置系统时间后，要使用“hwclock -w” 命令同步到 RTC；
② 使用 hwclock 查看、设置硬件时间，在设置硬件时间后，要使用“hwclock -s”命令同步到系统时间。

注意：如果要使用 RTC，要确保板子上已经安装了纽扣电池。

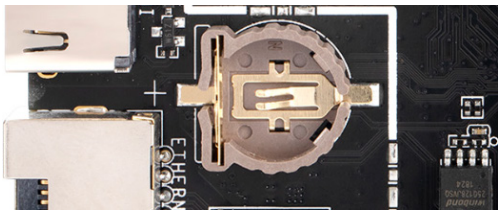


图 5.5.1 RTC 位置示意图

以下命令是设置系统时间、并同步到 RTC：

```
[root@roc-rk3399-pc:~]# date -s "2019-06-18 12:00:00"
[root@roc-rk3399-pc:~]# hwclock -w
```

```
[root@roc-rk3399-pc:~]# date -s "2019-06-18 12:00:00"
Tue Jun 18 12:00:00 UTC 2019
[root@roc-rk3399-pc:~]# hwclock -w
```

一般不直接设置硬件时间，要设置硬件时间时，先使用 date 设置系统时间，再使用“hwclock -w”同步到 RTC 硬件。

你使用 date、hwclock 命令设置好时间后，可以关闭开发板并等待一会后重启，再用 date 命令查看时间是否正常。

对 RTC 硬件的操作使用 hwclock 命令，常见用法如下：

- hwclock -r ：显示硬件时钟与日期
- hwclock -s ：将系统时钟调整为与目前的硬件时钟一致。
- hwclock -w ：将硬件时钟调整为与目前的系统时钟一致。

下面是一个示例，读取硬件时间：

```
[root@roc-rk3399-pc:~]# hwclock -r
Tue Jun 18 12:00:45 2019  0.000000 seconds
[root@roc-rk3399-pc:~]# hwclock -r
Tue Jun 18 12:00:51 2019  0.000000 seconds
[root@roc-rk3399-pc:~]# hwclock -r
Tue Jun 18 12:00:58 2019  0.000000 seconds
```

5.5 按键测试

此节演示通过 hexdump 命令以及 dmesg 命令来查看按键是否有反应。

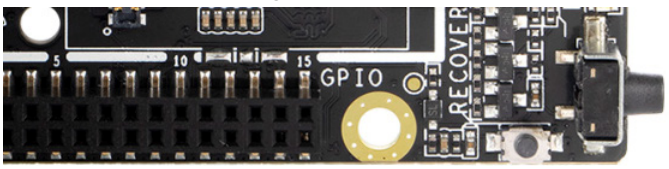


图 5.9.1 按键位置示意图

执行下面命令之后，操作按键，如果一切正常会有打印信息：

```
[root@roc-rk3399-pc:~]# hexdump    /dev/input/event2
```

```
[root@firefly-rk3288:~]# hexdump /dev/input/event2
0000000 1f3b 586a 950c 0001 0001 0074 0001 0000
0000010 1f3b 586a 950c 0001 0000 0000 0000 0000
0000020 1f3b 586a 52e2 0004 0001 0074 0000 0000
0000030 1f3b 586a 52e2 0004 0000 0000 0000 0000
0000040 1f3c 586a c0df 0008 0001 0074 0001 0000
0000050 1f3c 586a c0df 0008 0000 0000 0000 0000
0000060 1f3c 586a 9471 000a 0001 0074 0000 0000
0000070 1f3c 586a 9471 000a 0000 0000 0000 0000
```

5.6 LED 灯测试

此节演示通过操作 LED 在/sys 目录下的对应文件，以点亮、熄灭 LED。

注意： led1/led2/led3 灯已经被定义为系统状态灯，仍可在/sys 目录中操作对应文件；led4 留给用户用于其他目的。



图 5.7.1 led 灯位置示意图

首先，进入/sys/class/leds 目录查看在多少个 LED：

```
[root@roc-rk3399-pc:~]# cd /sys/class/leds/
```

```
[root@roc-rk3399-pc:~]# cd /sys/class/leds/
[root@roc-rk3399-pc:/sys/class/leds]# ls
firefly:red:power        firefly:yellow:heartbeat  firefly:yellow:user       mmc1::
```

这些文件对应的 LED 是：

- Led1　　firefly:red:power　红色电源状态灯
- Led2　　firefly:yellow:heartbeat　系统心跳灯
- Led3　　firefly:yellow:user　　用户灯

```
[root@roc-rk3399-pc:/sys/class/leds]# cd firefly\:yellow\:heartbeat/
[root@roc-rk3399-pc:/sys/devices/platform/leds/leds/firefly:yellow:heartbeat]# ls
brightness      device          max_brightness  power           subsystem       trigger         uevent
```

此时我们通过 echo 输入值到 brightness 即可控制灯的亮灭，以下以 firefly:yellow:heartbeat 为示例

```
[root@roc-rk3399-pc:/sys/devices/platform/leds/leds/firefly:yellow:heartbeat]# echo 1 > brightness    //灯亮
```

```
[root@roc-rk3399-pc:/sys/devices/platform/leds/leds/firefly:yellow:heartbeat]# echo 0 > brightness    //灯灭
```

5.7 串口测试

此节演示使用 linux-serial-test 程序测试串口的可用性。

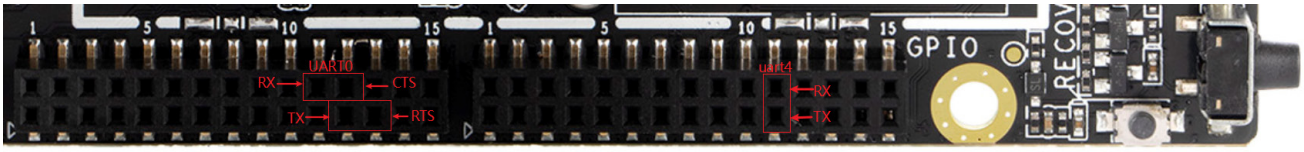


图 5.8.1 串口测试连接示意图

1) 压力测试：

这发送五秒钟并接收七秒钟，之后它将退出。如果接收的字节数与发送的字节数匹配且接收的模式正确，则退出代码将为零，因此可以将其用作自动测试脚本的一部分。

```
[root@roc-rk3399-pc:~]# linux-serial-test -s -e -p /dev/ttyS1 -b 115200 -o 5 -i 7
```

```
[root@100ask_am335x:~]# linux-serial-test -s -e -p /dev/ttyO2 -b 115200 -o 5 -i 7
Linux serial test app
Stopped transmitting.
/dev/ttyO2: count for this session: rx=57903, tx=57903, rx err=0
/dev/ttyO2: TIOCGICOUNT: ret=0, rx=730770, tx=767105, frame = 0, overrun = 0, parity = 0, brk = 0, buf_overrun = 0
Stopped receiving.
/dev/ttyO2: count for this session: rx=57903, tx=57903, rx err=0
/dev/ttyO2: TIOCGICOUNT: ret=0, rx=730770, tx=767105, frame = 0, overrun = 0, parity = 0, brk = 0, buf_overrun = 0
[root@100ask_am335x:~]#
```

2) 全速压力测试：

```
[root@roc-rk3399-pc:~]# linux-serial-test -s -e -p /dev/ttyS1 -b 3000000
```

```
[root@100ask_am335x:~]# linux-serial-test -s -e -p /dev/ttyO2 -b 3000000
Linux serial test app
```

参数详解

- -b， 串口波特率，115200 等（默认为 115200）
- -p， port Port（/ dev / ttyS0 等）（必须指定）
- -s， stats 每 5 秒转储串口数据
- -e， dump-err 显示错误
- -o， tx-time 要传输的秒数（默认为 0，表示没有限制）
- -i， rx-time 要接收的秒数（默认为 0，表示没有限制）